# Computational Modeling for Psychology Students

Britt Anderson

Winter 2026

## Contents

# 1 Welcome to Psych 420

## 1.1 Introductions assignment

1. Who am I?
2. Who are you? Send me an email (britt@uwaterloo.ca) right now (from your uwaterloo address) with
   (a) The subject line must be exactly P420Intro; nothing more; nothing less. Capitalization matters.
   (b) Your name (and any pronounciation tips).
   (c) Area of concentration,
   (d) Year of study,
   (e) Programming language you are most capable in (and your level of competency),
   (f) Reason you took this course.

3. What you will need: A functioning computer that you know how to use. What to do now:

    - Update your operating system.
    - Verify you know how to find a file on your computer (e.g. do you know the name of your home directory)?
    - Confirm you know how to install software on your computer.
    - You know (or will know by next week) what `git` is and how to use it, and that you will have an `IDE` installed.

## 1.2 What I am assuming you know or will learn on your own.

1. Your computer is up to date.
2. You know what a directory is and how to find files on your computer.
3. You know how to install software.
4. You have at least beginner level programming experience in one course.
5. You know what git/github are.

### 1.2.1 What if you don't?

You will have to learn on your own or with a small group. You can consult the course notes for Psych 390 which does cover some of this material. Here is a link to the wiki.

### 1.2.2 Git Clone and Github Acct assignment

An early home work will be a screen shot of your github homepage showing you forked or cloned the course repository. And the screen shot should be done using a program and not your phone. Course Webpage . That is the course content. If you want to view a web viewable form you can go to this link: course notes.

## 1.3 Some recommendations.

You will want an IDE. I personally use emacs. Experience has taught me that only about 3 in a 100 of you will share my opinion. Thus, I recommend VS Code if you have no other preference. It is becoming a sort of de facto standard.

#### 1.3.1 IDE Installed **assignment**

Send me a screen shot of the IDE you intend to use installed on your laptop. **No** phone cameras. Use the screenshot functionality of your computer.

### 1.4 What is computational cognitive (neuro)science?

1. Form groups of two (or three if odd number).
2. Go to the following web site: 2025 Posters Cognitive Computational Neuroscience meeting.
   (a) Pick a poster. Note it's label (letter-number; on the left)
   (b) Be ready with a two sentence summary.
   (c) Tell us why it is a good example of computational modelling of either mind or brain.
   (d) Why you chose it.

### 1.5 Syllabus Review

#### 1.5.1 Course number and title

PSYCH420 Introduction to Computational Neuroscience Methods

#### 1.5.2 Term and year of offering

Winter 2026

#### 1.5.3 Class days, times, building, and room number

Th 11:30 - 14:20 HH 2107

#### 1.5.4 Class instructor's name, office, contact information, office hours

Britt Anderson, PAS 4032, x 43056, Office hours by arrangement

#### 1.5.5 Teaching assistant's name, office, contact information, office hours (if applicable)

#### 1.5.6 Course description

An introduction to the basic elements of computational modeling with an emphasis on psychology students.

### 1.5.7 Course objectives

1. You will understand the scope of subjects and tools that comprise computational neuroscience and computational cognitive modeling.

2. You will have a better understanding of theory development and how it relates to ideas about computation, programming, mind and brain.

3. You will have a better understanding of how programming language choice influences model and theory development.

4. You will learn some of the fundamental notation and concepts of the mathematical domains that form the basis of much computational modeling in neuroscience and psychology.

5. You will gain experience in the practical aspects of programming simple models by doing.

### 1.5.8 Required text and/or readings

There is no required text book. Material that might be needed outside what I present in class will be made available on line, predominantly at the course repo on github. Make sure you are tracking the **W2026** branch.

As all University students should have access to a computer, and a computer is required for this course, there is $0 CAD additional expense.

### 1.5.9 A general overview of the topics to be covered

See Course objectives.

### 1.5.10 The evaluation structure for the course including course requirements, deadlines, weight of requirements toward the final course grade

| Category | Contribution |
|---|---|
| Participation | 12%. |
| Final Project | 20%. |
| Homeworks | 68% |

It is easy to find high quality material to read or watch. What good then is the classroom? The classroom is for doing and clarifying. You have no way of knowing if your understanding developed from online material is complete or accurate unless you match it off against others. Your peers and

your instructor. Thus, we shouldn't use class time for presenting material that can be read or watched anywhere anytime, but we should use it for testing whether your knowledge is sound, flexible, and deployable. We will therefore read, discuss, and collaborate.

Learning math and computing, and how to use them, is not something that can be learned through observation. Like playing piano, or improving your golf swing, improvement comes from practice, practice, practice. It is not enough to merely state the principles. You must demonstrate that they have been internalized. We will therefore do small exercises in class. And since one of the best ways to learn is to teach, we will frequently do these exercises in small groups. You might find your peers description of a problematic fact more useful than mine since they are at a more similar learning level. You may find your own understanding grows as you have to reframe my teachings in your own words to help a fellow student grasp a difficult concept.

For these reasons class participation is important. It is where we will test our comprehension of important points, I will learn if a concept is not well appreciated, and you will have the chance to practice the skills you later may wish to use in your own way on your own problems. Each week's attendance is worth 1% of the final grade.

I like to give lots of small homeworks. That takes the pressure of any one assignment, and means you get both frequent feedback on your performance, and you are coerced into not falling behind.

The final project is meant to give you an opportunity to showcase your learning and to use the general and specific skills that you will use in the future, whether in academics or industry, for sharing complex technical material with others.

### 1.5.11 Acceptable rules for group work (See Group Assignment Checklist)

Group work is the norm in post-graduate life (both academic and professional). Group work will be graded as a group. All group members will get the same grade for their group project. More details on the nature of the group project will be shared during the term.

### 1.5.12 Indication of how late submission of assignments and missed assignments will be treated

If I know that an assignment will be late in advance, and the reason for the tardiness is beyond your reasonable control, then I probably won't take off any points for late submission. Otherwise, I may reduce the possible maximal score by 10% per week.

### 1.5.13 Indication of where students are to submit assignments and pick up marked assignments

Assignments will be submitted in dropboxes on Learn. Submissions and Announcements are the main uses we will make of learn. All other material will be shared in class or via the github repository.

### 1.5.14 Intellectual Property

Intellectual property includes items such as:

- Lecture content, spoken and written (and any audio/video recording thereof);
- Lecture handouts, presentations, and other materials prepared for the course (e.g., PowerPoint slides);
- Questions or solution sets from various types of assessments (e.g., assignments, quizzes, tests, final exams); and
- Work protected by copyright (e.g., any work authored by the instructor or TA or used by the instructor or TA with permission of the copyright owner).

Course materials and the intellectual property contained therein, are used to enhance a student's educational experience. However, sharing this intellectual property without the intellectual property owner's permission is a violation of intellectual property rights. For this reason, it is necessary to ask the instructor, TA and/or the University of Waterloo for permission before uploading and sharing the intellectual property of others online (e.g., to an online repository).

Permission from an instructor, TA or the University is also necessary before sharing the intellectual property of others from completed courses with students taking the same/similar courses in subsequent terms/years. In many cases, instructors might be happy to allow distribution of certain

materials. However, doing so without expressed permission is considered a violation of intellectual property rights.

Please alert the instructor if you become aware of intellectual property belonging to others (past or present) circulating, either through the student body or online. The intellectual property rights owner deserves to know (and may have already given their consent).

### 1.5.15 Chosen/Preferred First Name

Do you want professors and interviewers to call you by a different first name? Take a minute now to verify or tell us your chosen/preferred first name by logging into WatIAM.

Why? Starting in winter 2020, your chosen/preferred first name listed in WatIAM will be used broadly across campus (e.g., LEARN, Quest, WaterlooWorks, WatCard, etc). Note: Your legal first name will always be used on certain official documents. For more details, visit Updating Personal Information.

Important notes

If you included a preferred name on your OUAC application, it will be used as your chosen/preferred name unless you make a change now. If you don't provide a chosen/preferred name, your legal first name will continue to be used.

### 1.5.16 Mental Health Support

All of us need a support system. The faculty and staff in Arts encourage students to seek out mental health support if they are needed.

1. On Campus

   Counselling Services: counselling.services@uwaterloo.ca / 519-888-4567 ext. 32655 MATES: one-to-one peer support program offered by the Waterloo Undergraduate Student Association (WUSA) and Counselling Services

2. Off campus, 24/7

   Good2Talk: Free confidential help line for post-secondary students. Phone: 1-866-925-5454 Grand River Hospital: Emergency care for mental health crisis. Phone: 519-749-4300 ext. 6880 Here 24/7: Mental Health and Crisis Service Team. Phone: 1-844-437-3247 OK2BME: set of support services for lesbian, gay, bisexual, transgender or questioning teens in Waterloo. Phone: 519-884-0000 extension 213

### 1.5.17 Territorial Acknowledgement

Academic freedom at the University of Waterloo We acknowledge that we are living and working on the traditional territory of the Attawandaron (also known as Neutral), Anishinaabe and Haudenosaunee peoples. The University of Waterloo is situated on the Haldimand Tract, the land promised to the Six Nations that includes ten kilometres on each side of the Grand River.

For more information about the purpose of territorial acknowledgements, please see the CAUT Guide to Acknowledging Traditional Territory. Academic freedom at the University of Waterloo

### 1.5.18 Policy 33, Ethical Behaviour

Policy 33, Ethical Behaviour states, as one of its general principles (Section 1), "The University supports academic freedom for all members of the University community. Academic freedom carries with it the duty to use that freedom in a manner consistent with the scholarly obligation to base teaching and research on an honest and ethical quest for knowledge. In the context of this policy, 'academic freedom' refers to academic activities, including teaching and scholarship, as is articulated in the principles set out in the Memorandum of Agreement between the FAUW and the University of Waterloo, 1998 (Article 6). The academic environment which fosters free debate may from time to time include the presentation or discussion of unpopular opinions or controversial material. Such material shall be dealt with as openly, respectfully and sensitively as possible." This definition is repeated in Policies 70 and 71, and in the Memorandum of Agreement, Section 6

## 1.6 A look ahead – what we will code

1. Turing Machine
2. Integrate and Fire Neuron
3. Hodgkin-Huxley Neuron
4. Perceptron
5. Hopfield Net
6. Agent Based Model
7. And possibly more

## 1.7 A look ahead – what we talk about

1. What is cognition?
2. What is a cognitive model?
3. What is the relationship between cognition and neuroscience?
4. All of which will benefit from better understanding what is a theory.

## 1.8 What is Cognition?

Groups of three. I prefer you not have the same people in your group as last time. We want to promote meeting people. It makes it easier for you to help each other, find people at your own learning level to study and work with, gauge your own skill level, develop informal networks that may be of use outside of this class. What is Cognition? Each group gets an opinion (want three of four groups) Brainard/Byrne/Heyes/Suddendorf 20 minutes to read and discuss to make sure they understand. Then rotate the groups so that each group has only one member/opinion. 15 minutes to explain the opinions to each other. General discussion.

## 1.9 Is a mathematical formula a model?

### 1.9.1 Forgetting Curve — Ebbinghaus

### 1.9.2 A Mathematical Function

```
import Graphics.Gnuplot.Simple
plotFunc [PNG "./images/expdecay.png"]
(linearScale 100 (1,6))
((\x -> exp (1.0/x)) :: Double -> Double)
```

1. A digression on code The above example is coded in Haskell. If any of you feel like python or R are getting a bit stale and you would like to learn a new language I encourage you to consider Haskell. I picked it for use here because,
   (a) I like it and want to get better at using it,
   (b) I did not think many (or any) of you would know it,
   (c) And therefore I could talk about coding a particular algorithm without ruining your opportunity to practice coding it yourself.

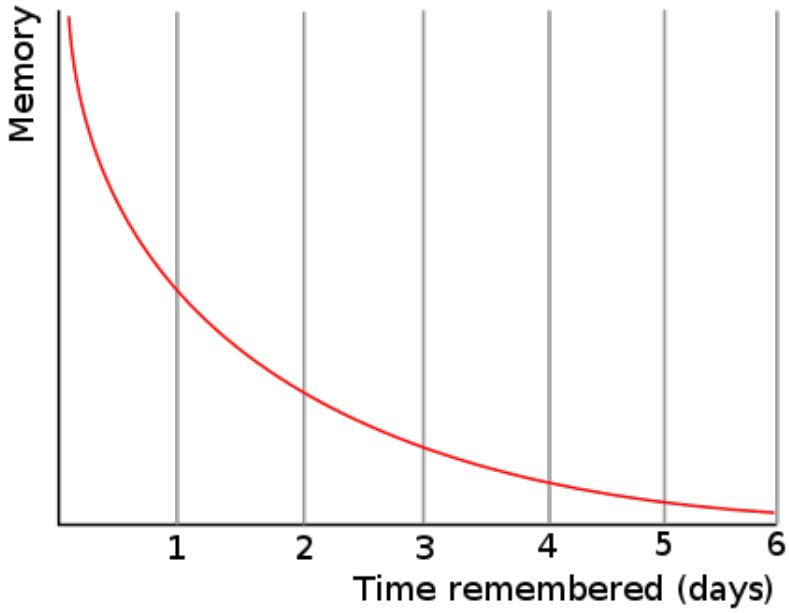



Figure 1: By The original uploader was Icez at English Wikipedia.

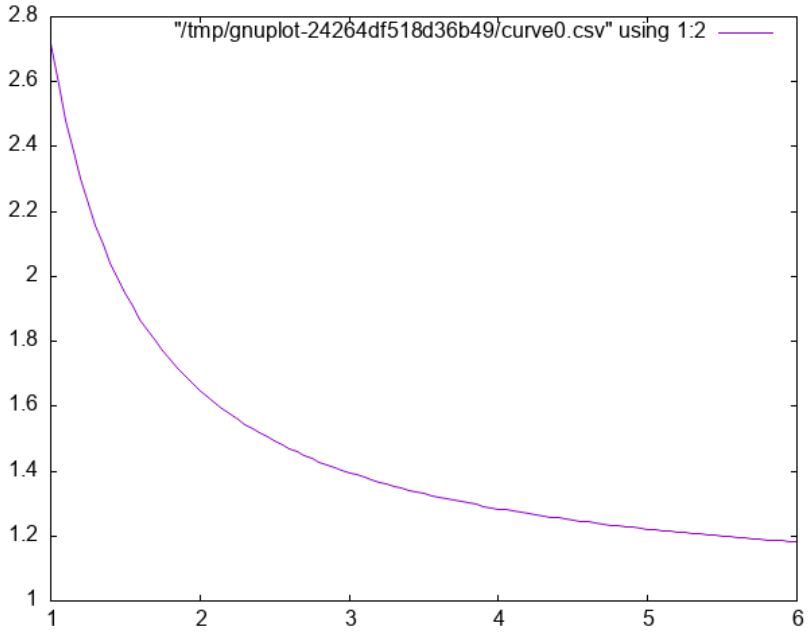



Figure 2: Plot of $e^{\frac{1}{x}}$.

But what about Large Language Models (LLMs)? They are great, and you should definitely use them, **but** use them deliberately and after reflection.

(a) Do *not* use them to do an exercise for you. You may learn something about LLMs, but you will *not* learn how to code that algorithm or very much about the programming language you want to master.

(b) *Do* use them to help your learning and understanding.

- You have written some code yourself and it is not working, and you can't figure out why. Ask your llm to find the bug *and* explain it.
- You want to know if your code is idiomatic. Ask your llm for a critique and comment on working code.

(c) If you have no idea how to get started *do* ask an LLM to architect without asking it to code.

(d) Do *not* live in the web/browser app. Get yourself an api key and access the LLM from your chosen IDE. Two tools that I use often

are aider and openrouter. The latter will give you access to free models. Aider will allow you to call the LLM from the terminal. In Emacs I find **gptel** extremely convenient.

This course is not to teach you how to use LLMs or set them up, but since many of us will be making use of them we should talk about the pros and cons of particular use cases and outputs. Treat your LLM usage like a citation. Say when and where you used it.

#### 1.9.3 What can we infer

from the similarity between Figure 1 and Figure 2 ?

#### 1.9.4 Generate your own plot of the exponential function. assignment

# 2 Neurally Inspired Models 1: The cellular level

## 2.1 Action Potentials

Our goals:

1. Understand what we are trying to model: the action potential.
2. Learn some of the prior work.
3. Understand the mathematical background for an action potential.
   (a) Simple electronics.
   (b) Simple differential equations.
4. Make ourselves familiar with the basic notation of these two background areas.
5. Implement simple computational versions of spiking neurons. If you cannot code it, you do not understand it.

## 2.2 What is an action potential?

1. Draw one.
2. Explain what ions account for the different parts of the curve.
3. Why is an action potential like a flush toilet?

4. Why did/do people use the action potential to argue that brains are like digital computers?
5. Who were McCulloch and Pitts and why are they relevant?
6. Who developed the first detailed biophysical model of the action potential and what became of them?
7. Why would we want to write a program to simulation a spiking neuron?
8. Why might we want a simpler version?
9. What maths are important for modeling spiking neurons?

## 2.3 Notation

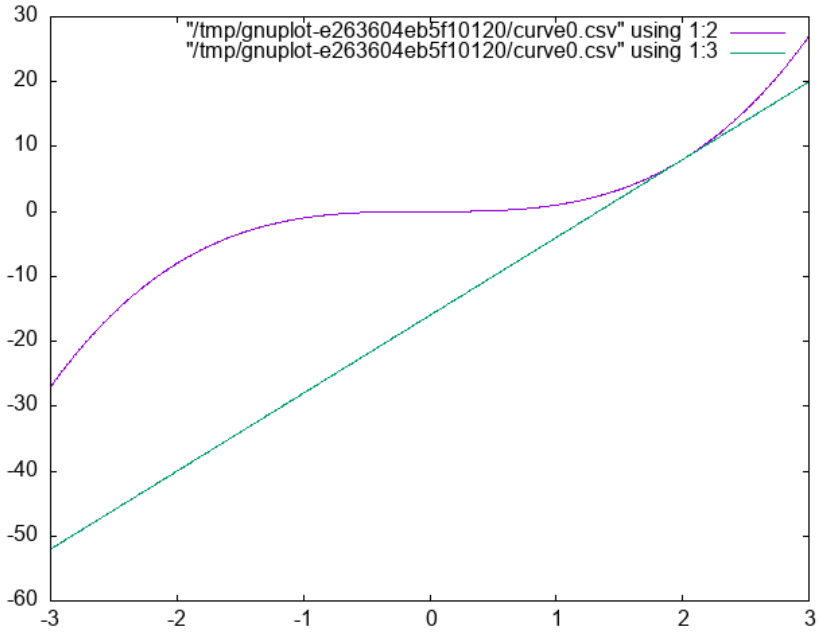
The mathematician's interest in being concise often means that simple ideas get translated to unfamiliar symbols. \ Just because the notation is unfamiliar does not mean the idea is complicated. What does $\sum_{\forall x \in \{1,2,3\}} x = 6$ mean?

Different mathematical and computational traditions have their own preferences, thus the same idea can have different notations. To a mathematician the $\sqrt{-1} = \texttt{i}$. To the engineer it is `j`. $\frac{dx}{dt}$ $\dot{x}$ $x'$ $f'(x)$ are all different ways of talking about derivatives.

## 2.4 Differential Equations

1. What is a differential equation (aka de)?
2. What is a derivative?
3. What is the **definition** of a derivative? $\frac{dx}{dy} = \frac{f(x+\Delta x)-f(x)}{x+\Delta x-x}$
4. What is a slope of a line?
5. Of a curve?
6. Provide examples of phenomena in psychology or neuroscience (other than the action potential) that might be good candidates for this mathematical technique.

```
plotFuncs [PNG "./images/derivExample.png"] (linearScale 1000 (-
3 , 3 ) )
  [(\x -> x ** 3) :: Double -> Double
  , (\x -> 12 * x - 16) :: Double -> Double ]
```




## 2.5 Using a DE

In computational neuroscience (or psychology) we don't typically " solve DEs. We use them. We use them to help us move forward in time to understand a process by virtue of what we know about its rate of change. We will see that in a couple of short examples, and then some simple scenarios for you to play with.

## 2.6 Using DEs to compute a square root

```
 1  module Sqrt where
 2
 3  xcubed :: Double -> Double
 4  xcubed  = (** 3)
 5
 6  diffXCubed :: Double -> Double
 7  diffXCubed = ( 3 *) . ( ** 2)
 8
 9  getStep :: Double -> Double -> Double
10  getStep guess goal = ( goal - xcubed guess)  / diffXCubed guess
11
```

```
12  getCubeRoot :: Double -> Double -> Double -> Double
13  getCubeRoot g i t =
14    let  cg = i + getStep i g
15         myerr = abs (xcubed cg - g) in
16      if myerr > t then getCubeRoot g cg t else cg
```

Now let's test it:

```
import Sqrt
getCubeRoot 27 1 0.001
getCubeRoot 8 1 0.001
getCubeRoot 141 1 0.001


3.0000005410641766
2.000004911675504
5.204827869200383
```

## 2.7 Using DEs - Practicing with Springs

Our Hodgkin-Huxley model will be will require us to use several different equations. And though the intergrate and fire neuron only requires one, a neuron is still rather abstract as a computational entity. Therefore, we will begin our DE coding practice with a simple, idealized spring.

Our differential equation is $\frac{d^2 position}{dt^2} = -P\ position$.

where 'position' refers to the location in space of the unattached end of our spring (we assume one end is anchored to a wall or something), 't' refers to time, and 'P' is a constant, often called the spring constant, that indicates how springy the spring is.

If we knew where the spring was now, we could use this equation to tell us where it would be a little bit into the future; just as you can use your velocity on the highway to estimate your location in the future. The smaller the time step the more accurate will be our estimate. This simple Euler method often works quite well for smooth functions that don't change very much.

Unfortunately the spring equation is for the **second** derivative of location relative to time. This is called acceleration. Thus we need a two step process. From our current location and this equation estimate our velocity. From our present location and our velocity estimate our location. And repeat.

```
1  {-# LANGUAGE InstanceSigs #-}
```

```
 2  module Spring where
 3
 4  data SpringState = SpringState {
 5    sDt :: !Double,
 6    sAccFxn :: !(Double -> Double),
 7    sVel :: !Double,
 8    sLoc :: !Double
 9  }
10
11  instance Show SpringState where
12    show :: SpringState -> String
13    show s = "SpringState { t: " ++ show (sDt s) ++ "\
14             \, v: " ++ show (sVel s) ++ "\
15             \, loc: " ++ show (sLoc s) ++ " }"
16
17
18  {- Initial Values -}
19
20  initV :: Double
21  initV = 0
22  initLoc :: Double
23  initLoc = 10
24  sprConst :: Double
25  sprConst = 2.0
26  timeStep :: Double
27  timeStep = 0.05
28  sprdt :: Double
29  sprdt = timeStep
30
31
32  {- Helper Functions -}
33
34  accel :: Double -> Double -> Double
35  accel sc loc = (-sc) * loc
36
37  accWithSC :: Double -> Double
38  accWithSC = accel sprConst -- called currying
39
40  eulerUD :: Double -> Double -> Double -> Double
41  eulerUD ts roc oldval = oldval + roc * ts
```

```
42
43  springLoop :: SpringState -> SpringState
44  springLoop ss =
45    let cA = sAccFxn ss (sLoc ss)
46        cV = eulerUD (sDt ss) cA (sVel ss)
47        cL =  eulerUD (sDt ss) cV (sLoc ss)
48    in ss {sVel = cV, sLoc = cL}
49
50
51  releaseSpring :: Int -> [SpringState]
52  releaseSpring maxIter =
53    take maxIter (iterate springLoop
54                  (SpringState timeStep
55                   accWithSC initV
56                   initLoc)) -- makes use of lazy evaluation

import Spring
--    :load "./src/Spring.hs"
plotList [PNG "./images/spring.png"]
      $ map (\s -> sLoc s) $ releaseSpring 2000
```

### 2.7.1 Now You Try with a Damped Oscillator

The changes to whatever code you have for the frictionless spring should require very little alteration to work with the following DE:

$$\frac{d^2s}{dt^2} = -P\ s(t) - k\ v(t)$$

## 2.8 Solving DEs

There are many well established methods for solving systems of differential equations. If you really need one you can look it up. As a psychologist *using* a DE you will almost never have to solve, which means in this context to compute an analytical solution (analytical means using symbols). If you decide you must, just guess, and guess an exponential. Most of the differential equations we will see, and that you will see in biology or psychology, have the dependent variable present both on the left and right sides of your differential equation. The exponential is almost always in there somewhere as $e^x$, since the derivative of $\frac{de^x}{dx} = e^x$.

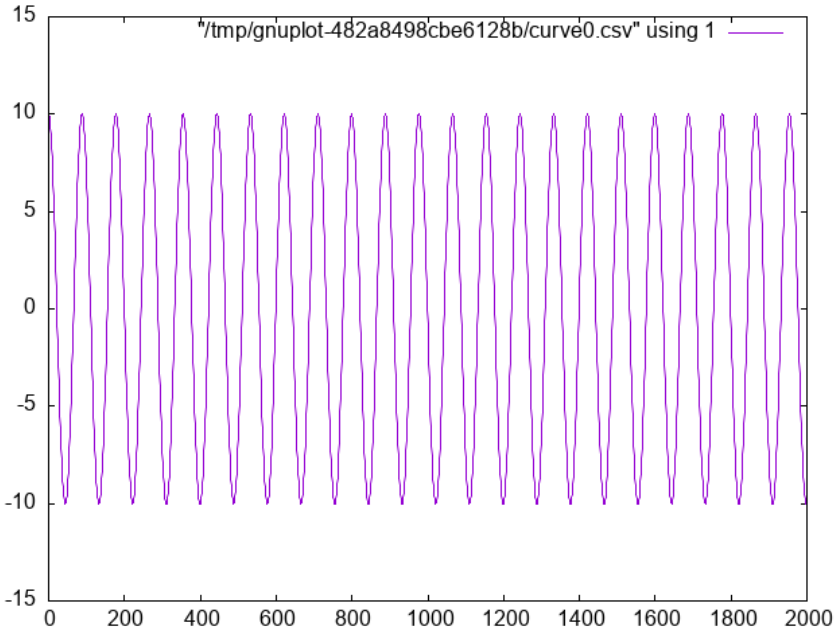



Figure 3: Frictionless Springs Oscillate

Try this with the undamped oscillator. We know that the second derivative of how position changes with time will eventually be a function of current position so we guess that $s(t) = e^{rt}$. Why the "r"? I know that the answer will have to be more than just $e^x$ so I throw in "r" for now assuming I can figure out the details later. Now, we act on our guess. If

$$s(t) = e^{rt} \quad \text{then} \tag{1}$$

$$\frac{d\ s(t)}{dt} = r\ e^{rt} \quad \text{and thus} \tag{2}$$

$$\frac{d^2 s(t)}{dt} = r^2 e^{rt} \quad \text{now plugging in our guess to our known formula we have} \tag{3}$$

$$r^2 e^{rt} = -P\ e^{rt} \quad \text{moving the right side to the left and factoring,} \tag{4}$$

$$e^{rt}(r^2 + P) = 0 \quad \text{we only need to worry about the right hand side so r must be,} \tag{5}$$

$$r = \pm i\sqrt{P} \quad \text{which means that are solution is,} \tag{6}$$

$$s(t) = e^{it} + e^{-it} \quad \text{and, thanks to Euler's } e^{i\Theta} = cos\Theta + i\ sin\Theta \text{ and simplifying we have,} \tag{7}$$

$$s(t) = 2cos\ t \tag{8}$$

## 2.9 Integrate and Fire Neuron

Is the integrate and fire model used much in modeling in the present time? answer

### 2.9.1 Historical Notes

1. Louie La Pique Modern Commentary on Lapique's Neuron Model

   Original Lapique Paper (translated) Brief Biographical Details of Lapicque

2. Lord Adrian

   (a) When was the action potential demonstrated?

   (b) What was the experimental animal used by Adrian?

   Adrian's Nobel Lecture Hodgkin's Memoir of Adrian

### 2.9.2 Formula I and F

$\tau \frac{dV(t)}{dt} = -V(t) + R\ I(t)$

Why is this the formula?


Figure 4: La Pique Laboratory at the Sorbonne Paris

1. Simple Electronics The following questions are the ones we need to answer to derive our integrate and fire model.

    (a) What is Ohm's Law?
    (b) What is Kirchoff's Point Rule?
    (c) What is Capacitance?
    (d) What is the relation between current and charge?

2. Deriving the IandF Equation

    Can you tell why, looking at the integrate and fire equation, if we don't reach the firing threshold, we see an exponential decay?

$$
\begin{aligned}
I &= I_R + I_C && (a)\\
&= I_R + C\frac{dV}{dt} && (b)\\
&= \frac{V}{R} + C\frac{dV}{dt} && (c)\\
RI &= V + RC\frac{dV}{dt} && (d)\\
\frac{1}{\tau}(RI - V) &= \frac{dV}{dt} && (e)
\end{aligned}
$$

a: Kirchoff's point rule,

b: the relationship between current, charge, and their derivatives

c: Ohm's law

d: multiply through by R

e: rearrange and define $\tau$

### 2.9.3 Coding up the Integrate and Fire Neuron

Approach this with the same ideas as the spring example. You have a certain initial value, and you update that using your differential equation that relates how much voltage changes as a function of time and current voltage.

Unlike real neurons the Integrate and Fire neuron model does not have a natural threshold and spiking behavior. You pick a threshold and everyone time your voltage reaches that threshold you designate a spike and reset the voltage.

**Additional Activities** (if that goes too quickly):

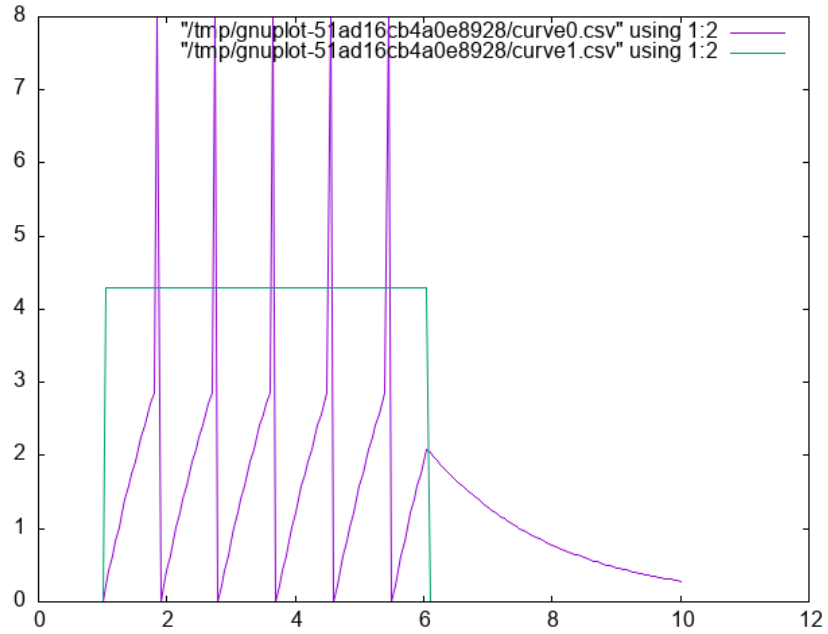
- Add a refractory period to the Integrate and Fire model.
- Alter the input current
- Change the form of the input current

```
 1  {-# LANGUAGE GADTs #-}
 2
 3  module IandF where
 4
 5  dt,maxt,initt,starttime,stoptime,cap
 6    ,res,threshold,spikedisplay,initv
 7    ,voltage,injectioncurrent
 8    ,iandftau :: Double
 9  injectiontime,runningTime :: [Double]
10  dt = 0.05
11  maxt = 10.0
12  initt = 0.0
13  starttime = 1.0
14  stoptime = 6.0
15  cap = 1.0
16  res  = 2.0
17  threshold = 3.0
18  spikedisplay = 8.0
19  initv = 0.0
20  voltage = initv
21  injectioncurrent = 4.3
22  injectiontime = [starttime, stoptime]
23  iandftau = res * cap
24  runningTime = [0.0]
25
26  data IandFStrut where
27    IandFStrut :: {
28      time :: [Double],
29      spikestatus :: Bool,
```

```
30      currents :: [Double],
31      voltages :: [Double]
32      } -> IandFStrut
33      deriving Show
34
35  instance Semigroup IandFStrut where
36    (IandFStrut t1 _ c1 v1) <> (IandFStrut t2 s2 c2 v2) =
37        IandFStrut (t1 ++ t2) s2 (c1 ++ c2) (v1 ++ v2)
38
39  initialStrut :: IandFStrut
40  initialStrut = makeStep 0 False 0.0 initv
41
42  update :: Double -> Double -> Double -> Double
43  update ov roc ts = roc * ts + ov
44
45  dvdt :: Double -> Double -> Double -> Double
46  dvdt lres li lv =
47   (1 / iandftau) * ( lres * li - lv )
48
49  between :: Double -> Double
50  between ct = case injectiontime of
51    [myfirst,mylast] ->
52      if ct >= myfirst && ct <= mylast
53      then injectioncurrent
54      else 0.0
55    badList -> error "Error: Expected list of two doubles"
56
57  vchoice :: (Double, Bool, Double, Double) -> Double
58  vchoice (cv,ss,thr,sd)
59    | ss = 0.0
60    | cv > thr  && not ss = sd
61    | otherwise = cv
62
63  makeStep :: Double -> Bool -> Double -> Double -> IandFStrut
64  makeStep t s c v = IandFStrut [t] s [c] [v]
65
66  oneRunIandF :: IandFStrut -> IandFStrut
67  oneRunIandF inStrut =
68    let ct = last (time inStrut)
69        cv = last (voltages inStrut)
```

```
70        ci = between ct
71        nv  = vchoice (update cv (dvdt res ci cv) dt
72                       , spikestatus inStrut
73                       , threshold, spikedisplay)
74        nextstep = makeStep (ct + dt)
75          (abs (nv - spikedisplay) < threshold)
76          ci
77          nv
78    in (inStrut <> nextstep)
```

```
  import IandF
--  :load ./src/IandF.hs
    let run200 = iterate oneRunIandF initialStrut !! 200
          vs = voltages run200
          cs = currents run200
          ts = time run200
          tv = zip ts vs
          tc = zip ts cs
    plotPaths [PNG "./images/iandf.png"] [tv,tc]
```



## 2.10 Integrate and Fire Homework

The Integrate and Fire homework has two components. One practical and one theoretical.

Practically, submit an integrate and fire program. Do something to show that you had something to do with the coding and you understand what is happening.

Theoretically, look at this article (pdf) and tell me how you feel our integrate and fire model compares to these actual real world spiking data when both are give constant input. What are the implications for using the integrate and fire model as a model of neuronal function?

## 2.11 Hodgkin and Huxley Model

### 2.11.1 Who were Hodgkin and Huxley

### 2.11.2 Model Description

Gerstner's (free) book's chapter on the HandH model.

1. Comments and Steps in Coding the Hodgkin Huxley Model

   Some introductory reminders and admonitions:

   The current going in to the cell is intended to represent what an electrophysiologist would inject in their laboratory setting, or what might be changed by the input from other neurons. The total current coming out of the neuron is the sum of the capacitance (due to the lipid bilayer), and the resistance (due to the ion channels). This is **Kirchoff's** rule implemented in the Hodgkin and Huxley model.

   Unlike the Integrate and Fire model, which lumps all the ionic currents together, the HandH model decomposes the conductance into three parts. You have a resistance *for each ion channel*. The rule for currents in parallel is to apply Kirchoff's and Ohm's laws realizing that they all experience the same voltage, thus the currents sum. The Hodgkin and Huxley model has components for Sodium (Na), Potassium (K), and negative anions (still lumped as "leak" l).

   $\sum_i I_R(t) = \bar{g}_{Na} m^3 hl(V(t) - E_{Na}) + \bar{g}_K n^4 (V(t) - E_K) + \bar{g}_L (V(t) - E_L)$

   $I_{tot} = I_r + I_C$

   $I_{tot} = \bar{g}_{Na} m^3 h(V(t)l - E_{Na}) + \bar{g}_K n^4 (V(t) - E_K) + \bar{g}_L (V(t) - E_L) + c\,\frac{dV}{dt}$

   If you rearrange terms you can get the $\frac{dV}{dt}$ on one side of the equation by itself.

$c\,\frac{dV}{dt} = I_{tot} - (\bar{g}_{Na} m^3 h(V(t) - E_{Na}) + \bar{g}_K n^4 (V(t) - E_K) + \bar{g}_L (V(t) - E_L))$

You cannot program what you don't understand. Don't start writing code too soon. Writing code won't make a poorly understood idea clearer. Though it can reveal that you don't understand what you thought you understood. It is often best to start sketching ideas on paper or a blackboard before you open up your IDE.

For instance,

- What are the $\bar{g}_*$ terms?
- What are the $E_*$ terms?
- What do m, n, and h represent?
- Where did these equations come from?

**Remember, it is differential equations all the way down.** You can use the same Euler method to patiently update each of your terms and then repeatedly loop between them. But recognize there are DEs at multiple levels. Each of the $m$, $n$, and $h$ terms are also changing and regulated by a differential equation. They are dependent on voltage, too. For example, $\dot{m} = \alpha_m(V)(1 - m) - \beta_m(V)m$.

**Recognize that**

- Each of the m, n, and h terms have their own equation of exactly the same form, but with their unique alphas and betas (that is what the subscript means).
- What does the V in parentheses mean?
- When the proteins of the ion channels were finally sequenced (decades later), what do you think was the number of sub-units that the sodium and potassium channels were found to have?

**Initial Assumptions**

If you allow $t \to \infty$, then $\frac{dV}{dt} = ?$

- You assume that it goes to zero; that is, you reach steady state. Then you can solve for some of the constants.
- Where do the constants come from?
- They come from experiments, and you use what you are given.
- Assume the following constants - they are set to assume a resting potential of **zero** not -70 mV (why doesn't this matter)?

- These constants also work out to enforce a capacitance of 1.

The "infinity" or "long time resting state" of the $m$, $n$, and $h$ values (that I refer to as $m_{inf}$ in my code and these notes) is of the form $(m/n/h)_{inf} = alpha(m/n/h)(v)/(alpha(m/n/h)(v)+beta(m/n/h)(v))$

2. Constants Several different sets of constants are used for this model in published versions. They are all, in one sense, the same, but this set make the programming easier. For the moment, ignore the units and use these numbers.

Table 1: Constant Values to Use for our Hodgkin-Huxley Model

| **Constant** | **Value** |
|---|---|
| ena | 115 |
| gna | 120 |
| ek | -12 |
| gk | 36 |
| el | 10.6 |
| gl | 0.3 |

**Warning** These constants are adjusted to make the resting potential 0 and the capacitance 1.0.

3. Alpha and Beta Formulas

$$\alpha_n(V_m) = \frac{0.01(10 - V_m)}{\exp\left(\frac{10-V_m}{10}\right) - 1}$$

$$\alpha_m(V_m) = \frac{0.1(25 - V_m)}{\exp\left(\frac{25-V_m}{10}\right) - 1}$$

$$\alpha_h(V_m) = 0.07 \exp\left(\frac{-V_m}{20}\right)$$

$$\beta_n(V_m) = 0.125 \exp\left(\frac{-V_m}{80}\right)$$

$$\beta_m(V_m) = 4 \exp\left(\frac{-V_m}{18}\right)$$

$$\beta_h(V_m) = \frac{1}{\exp\left(\frac{30-V_m}{10}\right) + 1}$$

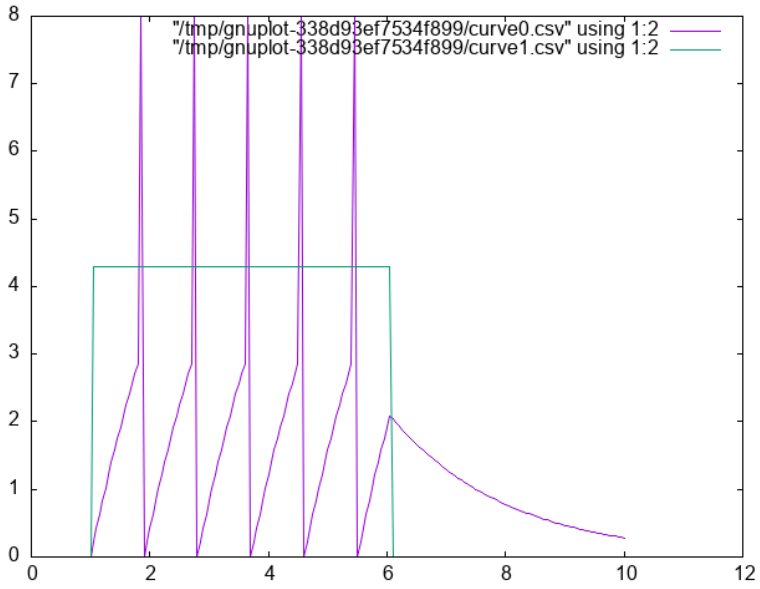
```
 1  {-# LANGUAGE GADTs #-}
 2
 3  module HandH where
 4  import Data.Maybe (fromMaybe)
 5
 6  data SimParameters where
 7    SimParameters :: {
 8      hhdt :: Float,
 9      maxT :: Float,
10      initT :: Float,
11      startTime :: Float,
12      stopTime :: Float,
13      capacitance :: Float,
14      resistance :: Float,
15      initialVoltage :: Float,
16      injectionCurrent :: Float,
17      hhtau :: Float
18      } -> SimParameters
19    deriving Show
20
21  data NeuronState where
22    NeuronState :: {
23      vns :: Float
24      , ic  :: Float
25      , neuTime :: Float
26      , mns :: Maybe Float
27      , nns :: Maybe Float
28      , hns :: Maybe Float
29    } -> NeuronState
30    deriving Show
31
32  data NeuronParams where
33    NeuronParams :: {
34      ena :: Float,
35      gna :: Float,
36      ek  :: Float,
37      gk  :: Float,
38      el  :: Float,
```

```
39      gl  :: Float
40      } -> NeuronParams
41    deriving Show
42
43  data NeuronDynamics where
44    NeuronDynamics :: {
45      alphaN :: Float -> Float,
46      alphaM :: Float -> Float,
47      alphaH :: Float -> Float,
48      betaN  :: Float -> Float,
49      betaM  :: Float -> Float,
50      betaH  :: Float -> Float,
51      mdot   :: Float -> Float -> Float,
52      ndot   :: Float -> Float -> Float,
53      hdot   :: Float -> Float -> Float,
54      minf   :: Float -> Float,
55      ninf   :: Float -> Float,
56      hinf   :: Float -> Float
57      } -> NeuronDynamics
58
59  data Neuron where
60    Neuron :: {parameters :: NeuronParams,
61               equations  :: NeuronDynamics
62              } -> Neuron
63
64  dotDynamics :: Float -> Float ->
65                 (Float -> Float) ->
66                 (Float -> Float) ->
67                 Float
68  dotDynamics volt  chan alphaf betaf =
69    alphaf volt * (1 - chan) - betaf volt * chan
70
71  dotInfinity :: Float ->
72                 (Float -> Float) ->
73                 (Float -> Float) ->
74                 Float
75  dotInfinity v alphaf betaf = alphaf v / (alphaf v + betaf v)
76
77  pSet1 :: SimParameters
78  pSet1 = SimParameters 0.05 300.0 0.0 100.0
```

```
79            150.0 1.0 2.0 0.0 20.0
80            (resistance pSet1 * capacitance pSet1)
81
82  pSet2 :: SimParameters
83  pSet2 = SimParameters 0.05 300.0 0.0 10.0
84             50.0 1.0 2.0 0.0 0.0
85             (resistance pSet2 * capacitance pSet2)
86
87  healthyParams :: NeuronParams
88  healthyParams = NeuronParams 115 120 (-12) 36 10.6 0.3
89
90  healthyDynamics :: NeuronDynamics
91  healthyDynamics = NeuronDynamics {
92    alphaN = \v -> (0.1-0.01*v)/(exp(1-0.1*v) - 1),
93    alphaM = \v -> (2.5-0.1*v)/(exp(2.5-0.1*v) - 1),
94    alphaH = \v ->  0.07*exp((-v)/20),
95    betaN  = \v ->  0.125 * exp((-v)/80),
96    betaM  = \v ->  4*exp((-v)/18),
97    betaH  = \v ->  1/(exp(3-0.1*v)+1),
98    mdot   = \v c -> dotDynamics v c (alphaM healthyDynamics)
99                          (betaM healthyDynamics),
100   ndot   = \v c -> dotDynamics v c (alphaN healthyDynamics)
101                         (betaN healthyDynamics),
102   hdot   = \v c -> dotDynamics v c (alphaH healthyDynamics)
103                         (betaH healthyDynamics),
104   minf   = \v -> dotInfinity v (alphaM healthyDynamics)
105                   (betaM healthyDynamics),
106   ninf   = \v -> dotInfinity v (alphaN healthyDynamics)
107                   (betaN healthyDynamics),
108   hinf  =  \v -> dotInfinity v (alphaH healthyDynamics)
109                   (betaH healthyDynamics)
110   }
111
112 healthyNeuron :: Neuron
113 healthyNeuron = Neuron healthyParams healthyDynamics
114
115 hhInitialState :: NeuronState
116 hhInitialState = NeuronState (initialVoltage pSet1)
117   (injectionCurrent pSet1)
118   (initT pSet1) Nothing Nothing Nothing
```

```
119
120 updNeuron :: Neuron  ->  SimParameters  ->  NeuronState ->  NeuronState
121 updNeuron neuin spin nsin =
122   let eqneuin = equations neuin
123       voltagein = vns nsin
124       nparamsin = parameters neuin
125       outi = ic nsin
126       mnew = fromMaybe 0.0
127         (case mns nsin of
128             Nothing ->  Just $ minf eqneuin  voltagein
129             Just x  ->  Just $ x +
130               mdot eqneuin  voltagein x * hhdt spin)
131       nnew = fromMaybe 0.0
132         (case nns nsin of
133             Nothing ->  Just $ ninf eqneuin  voltagein
134             Just x  ->  Just $ x +
135               ndot eqneuin voltagein x * hhdt spin )
136       hnew = fromMaybe 0.0
137         (case hns nsin of
138             Nothing ->  Just $ hinf eqneuin voltagein
139             Just x  ->  Just $ x +
140               hdot eqneuin voltagein x * hhdt spin)
141       dvdt = outi -
142         (gna nparamsin * mnew ^ 3 * hnew *
143           (voltagein - ena nparamsin) +
144           gk nparamsin * nnew ^ 4 *
145           (voltagein - ek nparamsin) +
146           gl nparamsin *
147           (voltagein - el nparamsin))
148   in
149     nsin {vns = voltagein + dvdt * hhdt spin
150          , ic = if (neuTime nsin  < stopTime spin) &&
151                    (neuTime nsin  > startTime spin)
152                 then injectionCurrent spin
153                 else 0.0
154          , neuTime = neuTime nsin + hhdt spin
155          , mns = Just mnew
156          , nns = Just nnew
157          , hns = Just hnew
158          }
```

```
import HandH
let myData = take 5000 $ iterate
      (updNeuron healthyNeuron pSet1 )
      initialState
    vs = map vns myData
    cs = map ic myData
    ts = map neuTime myData
    tv = zip ts vs
    tc = zip ts cs
    plotPaths [PNG "./images/handh.png"] [tv,tc]
```




5. More hints on programming the Hodgkin-Huxley model.

   (a) Work inside out By this I mean that you start with small functions that you know you will need and only later build out to chaining them altogether. It is easier to test individual functions.

   (b) Keep it working This means you test your code changes frequently and often. You don't want to miss a typo that breaks your code after two hours of all sorts of other changes.

   (c) Use the terminal The way to test is to write your code so that you can source it (or import it) as you work and test your code in

your interpreter. This is a lite-version of what is called a REPL: read-eval-print-loop.

(d) Use git (or something) By committing often you can more easily jump back to a working version if you break things horribly.

(e) Test with numbers where you know the answer. If you inject no current your voltage should not change. Because of small numerical imprecisions there may be some small non-zero numbers, especially early in your run, but you should definitely not drift off to larger positive or negative numbers.

(f) Code in a way that makes it easy to visualize variables We are often better at seeing what is going on when we look at a plot rather than a long list of numbers. By writing in a style that allows you to plot your variables you may be better able to debug errors.

(g) Expect that you will misplace a decimal. That is, don't get too discouraged. We all do this. At this stage I encourage you *not* to use a large language model to help you figure out the logic of your code, but if you think you have nailed the logic and just have a small typo or syntactic error somewhere, then by all means let the eagle eye of the chat bot help you find it. The more specific you are with your prompt, the better will the chat bot be able to educate. Don't say: "Why doesn't this work?". Do say: "I think there is a bug somewhere in the xyz function. Do you see a typo or similar bug in there?"

6. Hodgkin and Huxley Homework

   Your homework will require you to write the code for a Hodgkin-Huxley Neuron and then modify it. You will provide at least two plots in addition to your code. You will need to adapt the model to fit this paper.[1] These authors analyzed a series of inherited channelopathies (where ion channels are altered due to mutation) via simulation. In these conditions empirical measurements had been made on actual patients. The authors adapted the Hodgkin and Huxley model to permit them to explore the effects of such altered conductance on

[1] Omar A. Hafez and Allan Gottschalk, "Altered Neuronal Excitability in a Hodgkin-Huxley Model Incorporating Channelopathies of the Delayed Rectifier Potassium Channel," *Journal of Computational Neuroscience* 48, no. 4 (October 2020): 377–86, `https://doi.org/10.1007/s10827-020-00766-1`.

neuronal excitability. For this home work you will need to adapt the $n$ equation to be as follows:

$\frac{dn}{dt} = \gamma_\tau(\gamma_\alpha\alpha_n(v - \Delta V)(1 - n) - \gamma_\beta\beta_n(v - \Delta V)n)$

Then you must select one of the sets of $\gamma$ from their Table 1 and plot the neuronal activity.

To start with after you have implemented the new $n$ channel function set all the $\gamma$ s to 1. This should work exactly like the old model. Generate a plot showing that it does. For the second plot generate the same plot with your altered $\gamma$ values.

# 3 What is computation and what is its role in cognitive science?

We will be having a discussion on this. It only works if you have read the material that we want to discus. You need to read several sections of the The Computational Theory of Mind **before** class. These are all of section 3, 4.4, and all of 5 and 7. Feel free to read more. Can you answer the following questions after your reading? These are the sort of things we will be discussing.

## 3.1 What do you think?

1. Computational modeling of cognition and neural systems is/is-not the same as programming.
2. What else could computational modeling be?
3. What are we losing when we treat our computational model as a programmatic black box?
4. Is the computation that is cognitive the same as the computation that takes place in digital computers?
5. If so, why is there more than one programming language?
6. Might our choice of programming language based on features actually influence what we can conceive for a computational model?
7. What does it mean for something to be a model?
8. Is simulation good enough?

9. Is prediction good enough?

10. Do we want a model to *explain* something? And then of course what constitutes explanation?

## 3.2 Computing in Minds and Computers

One definition of whether something is computable is whether there exists a Turing Machine that can compute it. Although most of us learned the name "Turing" in the context of whether a computer has human intelligence, the so-called *Turing test*, the model of Turing computability emerged from thinking about humans computing[2]

## 3.3 What is a Turing machine? Some Background and Details

Turing machines are quite simple implements. While one can build a physical Turing machine, the more usual sense of the term is for a hypothetical computer that is comprised of:

1. a finite alphabet,

2. a finite set of states,

3. the capacity to read and write to a single location in memory, and the ability to adjust the memory location immediately left or right or to make no move at all, and

4. a set of instructions (or "machine table") that translates the combination of the current state and the current symbol to a new state and one of the acceptable actions.

### 3.3.1 Classroom Activity

Let's develop pseudo-code for the implementing a Turing Machine. It may help you when you do have to program a Turing machine.

[2]Alan Turing, "On Computable Numbers, with an Application to the Entscheidungsproblem (1936)," in *The Essential Turing* (Oxford University PressOxford, 2004), 58–90, https://doi.org/10.1093/oso/9780198250791.003.0005.

### 3.4 Programming a Turing Machine: the Busy Beaver

In my experience the only way to really come to grips with what a Turing machine is is to program one yourself. For this exercise we will program a Turing machine to solve an elementary version of the Busy Beaver problem. This is a famous problem, because it is easy to state and woefully difficulty to answer. In fact it is incomputable.

The following is a slightly formatted version of the Wikipedia description of a Turing machine.

1. A Turing machine has n "operational" states plus a Halt state, where n is a positive integer, and one of the n states is distinguished as the starting state.

2. The machine uses a single two-way infinite (or unbounded) tape.

3. The tape alphabet is {0, 1}, with 0 serving as the blank symbol.

4. The machine's transition function takes two inputs:

   (a) the current non-Halt state,

   (b) the symbol in the current tape cell,

   (c) and produces three outputs:

      i. a symbol to write over the symbol in the current tape cell (it may be the same symbol as the symbol overwritten),

      ii. a direction to move (left or right@";" that is, shift to the tape cell one place to the left or right of the current cell), and

      iii. a state to transition into (which may be the Halt state).

We will use it to guide us to write a simple instance that computes a solution to the Busy Beaver problem. A n-state Turing machine has $(4n + 4)2n$states. The formula is (symbols $\times$ directions $\times$ (states + 1))(symbols $\times$ states). The transition function (how to figure out where to go next may be seen as a finite look-up table. Each row of the table is a 5-tuple: (current state, current symbol, symbol to write, direction of shift, next state). Our goal with the Busy Beaver problem is to run our machine to produce as long a series of uninterrupted ones as we can and then **halt**.

What it means to "run" a Turing machine is to start in the starting state with the current tape cell being any cell of a blank (all-0) "tape", and then iterate the transition function. If the Halt state is entered then the number of 1s remaining on the tape is called the machine's score. Different

transition rules will give us different outputs, so we can score our machine based on its performance.

To restate in a more general way, the n-state busy beaver (BB-n) game is a contest to find an n-state Turing machine having the largest possible score — the largest number of 1s on its tape after halting. A machine that attains the largest possible score among all n-state Turing machines is called an n-state busy beaver, and a machine whose score is merely the highest so far attained (perhaps not the largest possible) is called a champion n-state machine.

### 3.4.1 A Busy Beaver Warm-Up

A simple version of the Busy Beaver problem, and one you can do by hand with pencil and paper, is the n=2 version. Create a Turing Machine with the following transition rules:

Here is the transition table for our machine:

| Machine Tuple In | Machine Tuple Out |
|---|---|
| a0 | b1r |
| a1 | b1l |
| b0 | a1l |
| b1 | h1r |

### 3.4.2 Busy Beaver Problem Demo Code

```
 1  module BusyBeaver where
 2
 3  import Data.Set (Set,fromList)
 4  import qualified Data.Map.Lazy as Map
 5  import Data.Map.Lazy (Map)
 6
 7  type TMState = Char
 8  type Position = Int
 9  data BinNum = Zero | One deriving (Show, Eq, Ord, Enum)
10  type TMTape = Map Int BinNum
11
12  data TuringMachine = TM
13    { state :: TMState
14    , tape  :: TMTape
15    , headLoc :: Int} deriving Show
16
```

```
17  testStates :: Set TMState
18  testStates = fromList ['a','b','h']
19
20  initialTape :: Map Int BinNum
21  initialTape = Map.empty
22
23  defaultTapeCell :: BinNum
24  defaultTapeCell = Zero
25
26  initialState :: TMState
27  initialState = 'a'
28
29  initialTM :: TuringMachine
30  initialTM = TM initialState initialTape 0
31
32  readFromTape :: TMTape -> Int -> BinNum
33  readFromTape t i =
34    Map.findWithDefault Zero i t
35
36  writeToTape :: BinNum -> Int ->  TMTape ->  TMTape
37  writeToTape bn hl =  Map.insert hl bn
38
39
40  updTM :: TuringMachine -> TuringMachine
41  updTM tm@(TM st t hl) =
42    let bn = readFromTape t hl in
43    case st of
44      'a' -> case bn of
45                Zero -> tm {state = 'b'
46                           , tape = writeToTape One  hl t
47                           , headLoc =  hl + 1}
48                One  -> tm {state = 'b'
49                           , tape = writeToTape One  hl t
50                           , headLoc =  hl - 1}
51      'b' -> case bn of
52                Zero -> tm {state = 'a'
53                           , tape = writeToTape One  hl t
54                           , headLoc =  hl - 1}
55                One  -> tm {state = 'h'
56                           , tape = writeToTape One  hl t
```

```
57                            , headLoc =  hl }
58      'h' -> tm
59      _ -> error "Entered incorrect state variable"
```

```
import BusyBeaver
--:load ./src/BusyBeaver.hs
let d = iterate updTM initialTM
tape $ d !! 7


fromList [(-2,One),(-1,One),(0,One),(1,One)]
```

1. Busy Beaver Homework See Dropbox on Learn

## 3.5 The Lambda Calculus

Turing machines are probably the most common framework used for asserting that cognition is computation, but there are others. Another widely used construction, especially in theoretical computer science, is the **lambda calculus**. Although these two constructions appear quite different on the surface, it has been shown that any thing that can be computed in one can be computed by the other. They are equivalent. This has led to a *conjecture* that anything computable can be computed by a Turing Machine or the Lambda Calculus (and some other additional, equivalent constructions).

### 3.5.1 Why should a psychology student care?

Because if this is true: than anything computable can be computed by the lambda calculus; *and* that the mind is basically a computation, then the mind can be computed by the lambda calculas. Everything that is mental can be accomplished by this single construct of the function and application.

### 3.5.2 Lambda Notation Basics

The lambda calculus is built entirely from functions.

1. Syntax
   (a) `\x.x` means "a function that takes input x and returns x" (the identity function) How might we right this in python? `def f (x): x` Note that the lambda calculus version is anonymous. It does not have an f. So, we really should write `(lambda x : x) (5)`

(b) `\x.y` means "a function that takes input x and returns y" (a constant function)

(c) Application: `(\x.x) 5` means "apply the identity function to 5", which gives us `5`

Try evaluating these by hand:

(a) `(\x.x) 7` evaluates to `7`

(b) `(\x.5) 9` evaluates to `5` (the input 9 is ignored).

(c) `(\x.x x) 3` evaluates to `3 3` (we apply 3 to itself… which doesn't make sense, but syntactically this is what happens)

2. The Basic Combinators

**I - Identity** `I = \x.x` Takes its input and returns it unchanged. Example: `I 5 → 5`

**K - Constant (Kestrel [3])** `K = \x.\y.x` Takes two inputs and returns the first, ignoring the second. Example: `K a b → a` Think of this as "pick the first argument."

**S - Substitution (Starling)** `S = \x.\y.\z.(x z)(y z)` This one is trickier. It takes three inputs and applies the first to the third, then applies the second to the third, then applies those results to each other. Example trace for `S K K x`:

(a) `S K K x`

(b) `(K x)(K x)` (by definition of S)

(c) `x` (because K x returns a function that ignores its input and returns x, so (K x)(anything) = x)

Notice: we just built the identity function I from K and S!

3. A Non-Terminating Example Here's something interesting: `(\x.x x)(\x.x x)`

Try to evaluate it:

(a) `(\x.x x)(\x.x x)`

(b) Applying the left function to the right: `(\x.x x)(\x.x x)`

(c) We're back where we started!

[3] Why the bird names? See To Mock a Mockingbird

This is an infinite loop, but we got it using only function application. No `while` loops, no recursion keyword - just functions.

Why might this be psychologically instructive? Can you think of any psychological or cognitive processes that might map on to this? Rumination? Sustained attention? Flow?

4. Write a python or R implementation of each of I, K, and S. Try running this code. Verify that:

   - `I(5)` returns `5`
   - `K(3)(7)` returns `3`
   - `S(K)(K)(5)` returns `5` (it acts like the identity function)

## 3.6 The Church-Turing Thesis

The two combinators (K, S) are sufficient to compute **anything** that your Turing machine can compute (but first, why don't I need to include I in this list?).

You can build:

- Numbers
- Arithmetic operations
- Conditional logic
- Recursion
- All computable functions

How would you use these *things* to make numbers?

### 3.6.1 Church Numerals

Church numerals show how numbers emerge from pure function structure—nothing but lambdas.

1. The Encoding

   A number n is represented as a function that applies some function f to an argument x exactly n times:

   0 = f. x. x 1 = f. x. f x 2 = f. x. f (f x) 3 = f. x. f (f (f x))

   Big woop? or Big woop!

Everything is "just" functions. Even numbers. If the brain (mind) is just a computation are numbers merely an encoding of how many times to do something?

### 3.6.2 Computing with the Lambda Calculus

Can you figure out what addition with Church Numerals looks like?

1. The Human Combinator Game

   (a) Setup Groups of 3-6. Each group designates:

   - One S person (green card)
   - One K person (red card)
   - One I person (yellow card)
   - Two or three Data people (blue cards with letters)

   Distribute role instruction cards. Ask each group member with a colored card to write down their "role" on the back legibly (so I can re-use them in a future year).

   (b) Challenge Rounds

   i. Round 1
   TARGET OUTPUT: b
   YOU HAVE: data values a and b
   TASK: Arrange combinators and data so the final result is b (not a).

   ii. Round 2
   TARGET OUTPUT: a
   YOU HAVE: data values a
   TASK: Use at least two combinators, but still end up with just a.

   iii. Round 3
   TARGET OUTPUT: a
   YOU HAVE: combinators S, K, I and data value a
   TASK: Achieve the same result as (I a) but using only S and K—no I.

   iv. Round 4
   TARGET OUTPUT: b a
   YOU HAVE: data values a and b, which can themselves act like functions
   TASK: Produce "b applied to a" as your result.

v. Round 5
TARGET OUTPUT: a a
YOU HAVE: data value a
TASK: Duplicate your input—end with two copies of a applied to each other.

### 3.6.3 Lambda Calculas Homework

1. HW 1: Computational Analogy Essay

   Choose one psychological process from the list below (or propose your own with instructor approval):

   - Memory retrieval
   - Selective attention
   - Emotional regulation
   - Decision-making under uncertainty
   - Language comprehension
   - Habit formation
   - Cognitive dissonance resolution

   Write an essay exploring this question: **If this psychological process were built from combinator-like primitives, what might those primitives be doing?**

## 3.7 Two Completely Different Models

| Turing Machine | Lambda Calculus |
|---|---|
| Tape, head, states | Functions only |
| Symbols and movement | Application and substitution |
| Physical/mechanical intuition | Abstract/mathematical |
| **Same computational power** | **Same computational power** |

This equivalence is one of the deepest results in computer science. It suggests that "computation" is a fundamental concept that transcends any particular implementation.

### 3.8 Implications

If anything that is computable can be computed by one of these systems,
and if thinking **is** computation,
then you can think with a Turing machine, and since Turing Machines can be built from non-biological parts then thinking is not just for people anymore.

# 4 Neurally Inspired Models 2: The network effect

## 4.1 Neural Networks

- What is a neural network?
- What mathematics are needed to build a neural network?
- How can neural networks help us understand cognition?

### 4.1.1 Cellular Automata

1. Drawing Exercise
   (a) Pick a number between 0 and 255 inclusive. Tell it to me when I ask.
   (b) Compute your rule. You write the configurations on top, and you write the base two representation of your number below. Like this for rule number 110.

| 111 | 110 | 101 | 100 | 011 | 010 | 001 | 000 |
|---|---|---|---|---|---|---|---|
| 0 | 1 | 1 | 0 | 1 | 1 | 1 | 0 |

   (c) Using your rule and your piece of graph paper start implementing your rule after coloring a single square in the middle of the top row "black" (equals 1).
   (d) Drop down one row and working from left to right color in the squares based on the three squares in the row above them.
   (e) The idea is something like this:
   (f) Do this for several rows.

**What you probably noticed**

This process is tedious and mistake prone. One good reason for translating your theoretical ideas into code is so that you are not tripped

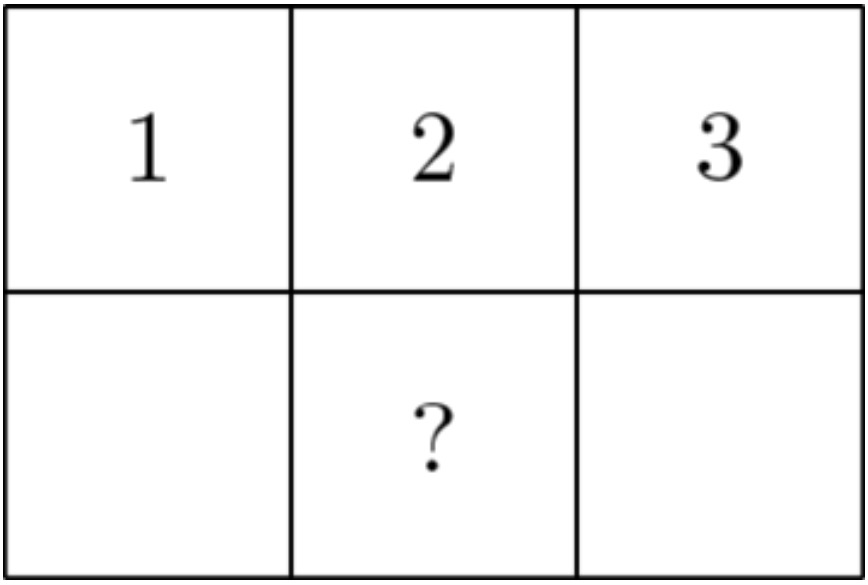



Figure 5: An illustration of which three squares in the row above influence the coloring of your grid.

up by such rote errors. Another thing to notice is that your "rule" is like a computer function. There is an input (the colors of the three squares above), processing, and an output of what color. In addition to this mechanical intuition a function can also be conceived as a set of pairs. The first element of the pair is the input, and the second element of the pair is the output.

```
import Automata
drawAutomata 30 128 5 "./images/rule30.png"
```

Figure 6: Invocation of Haskell Code for Generating a PNG file of Rule 30

**Lessons Learned**

- Repetitive actions are hard. We (humans) make mistakes following even simple rules for a large number of repeated steps. Better to let the computer do it since that is where its strengths lie.
- Complex global patterns can emerge from local actions. Each neuron is only responding to its immediate right and left yet global structures emerge.

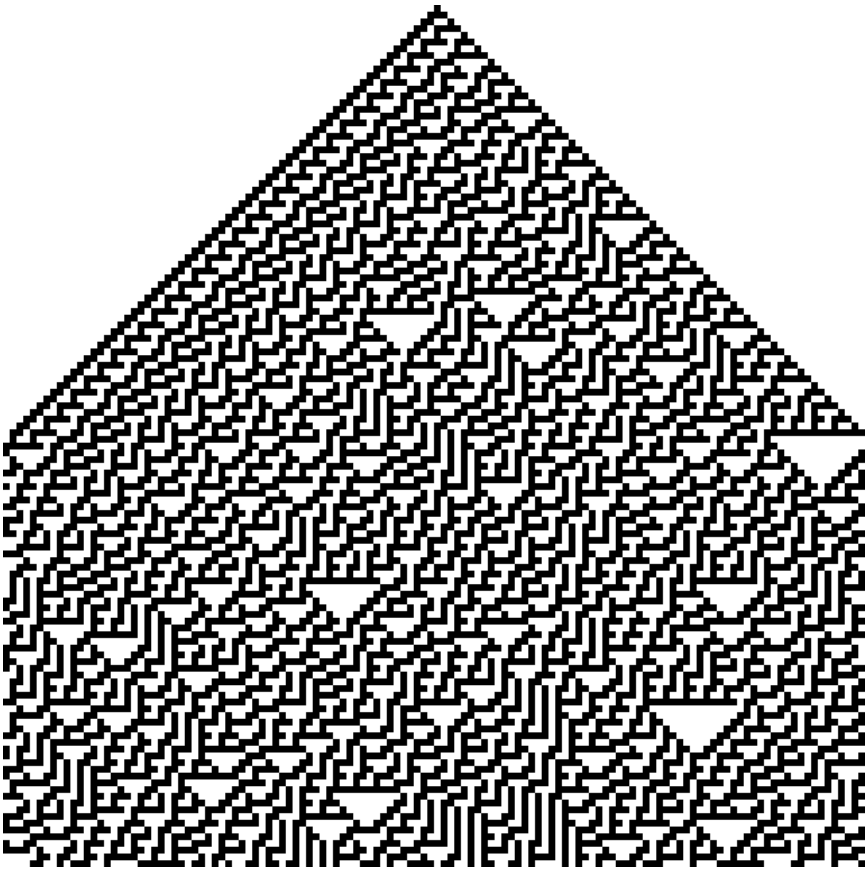


Figure 7: Rule 30

- These characteristics seem similar to brain activity. Each neuron in the brain is just one of many. Whether a neuron spikes or not is a consequence of its own state and its inputs (like the neighbors in the grid example).
- From each neuron making a local computation, global patterns of complex activity can emerge.
- Maybe by programming something similar to this system we can get insights into brain activity.

  There are common lisp, racket, versions of this code. Can you write your own? Can you write code to automate your rule?

### 4.1.2 More Lessons from Cellular Automata

Complex entities may have simple representations if the right language for expression can be found. Concision and repeatability are by-products.

There are also more dynamic versions of this idea. The "Game of Life" is one of the more famous.

Following up on the analogy to neurons, one humanity's most brilliant examples: John von Neumann, was working on a book (see also the commentary) about automata and the brain at the time of his death.

### 4.1.3 John Hopfield

To motivate our learning, and to put a human face on the project these two links give you a glimpse of Hopfield talking about his work at the 2024 Nobel Prize lectures, and a nice long podcast that gives a slow, throurough, tutorial discussion of what it is all about. We will come back to this later, but for now start listening. There is an associated homework.

1. Nobel Prize John Hopfield's Nobel Lecture

2. Podcast on the Hopfield Network This theoretical neuroscience podcast gives a nice long look at the Hopfield Network. It works up to its creation, its abstraction, and it social impact on shaping the field of computational neuroscience. It is long, but, if you make your way through it you will have a good overview of the model before you start coding it.

### 4.1.4 Linear Algebra

Show me the math!

The math at the heart of neural networks and their computer implementation is **linear algebra**. For us, the section of linear algebra we are going to need is mostly limited to vectors, matrices and how to add and multiply them.

**Discussion Question:**

What makes linear algebra linear?

1. Important Objects and Operations

   - Vectors
   - Matrices
   - Scalars
   - Addition
   - Multiplication (scalar and matrix)
   - Transposition
   - Inverse

   **Class Activity**
   Give a definition or description of each of these objects and operations.

   Most programming languages have specialized functions for defining and operating on matrices. I am using lists for ease and clarity, but for programming on larger, non-trivial, instances you will want to use your languages optimized linear algebra libraries.

```
1  {-# OPTIONS_GHC -Wno-unused-matches #-}
2  module MyMats where
3
4  mata :: SimpMat Integer
5  mata = [[1,2,3],[4,5,6]]
6  matb :: SimpMat Integer
7  matb = [[-1,-2,-3],[4,5,6]]
8  matc :: SimpMat Integer
9  matc = [[1,2],[3,4],[5,6]]           -- 3x2 matrix
```

```
10  matd :: SimpMat Integer
11  matd = [[7,8,9],[10,11,12]]          -- 2x3 matrix
12  mate :: SimpMat Integer
13  mate = [[1,0],[0,1]]                 -- 2x2 identity matrix
14  matf :: SimpMat Integer
15  matf = [[2,3],[4,5]]                 -- 2x2 matrix
16  matg :: SimpMat Integer
17  matg = [[1]]                         -- 1x1 matrix
18  math :: SimpMat Integer
19  math = [[1,2,3]]                     -- 1x3 matrix (row vector)
20  mati :: SimpMat Integer
21  mati = [[1],[2],[3]]                 -- 3x1 matrix (column vector)
22  matj :: SimpMat Integer
23  matj = [[0,0],[0,0]]                 -- 2x2 zero matrix
24
25
26  type SimpMat a = [[a]]
27
28  prettyMat :: Show a => SimpMat a -> String
29  prettyMat m = unlines $ map show m
30
31  add2SimpMats :: Num a => SimpMat a -> SimpMat a -> SimpMat a
32  add2SimpMats = zipWith (zipWith (+))
33
34  rotateLList :: SimpMat a -> SimpMat a
35  rotateLList [] = []
36  rotateLList l | any null l = []
37  rotateLList l = fmap head l : rotateLList (map tail l)
38
39  vectMult :: Num a => SimpMat a -> SimpMat a -> SimpMat a
40  vectMult v1 v2 =
41    let v1' = if length v1 == 1 then v1 else rotateLList v1
42        v2' = if length v2 == 1 then rotateLList v2 else v2
43    in multSimpMats v1' v2'
44
45  multSimpMats :: Num a => SimpMat a -> SimpMat a -> SimpMat a
46  multSimpMats m1 m2
47    | null m1 || null m2 =
48      error "Cannot multiply empty matrices"
49    | cols1 /= rows2 =
```

```
50      error $ "Dimension mismatch: " ++
51      show cols1 ++ "   " ++ show rows2
52    | otherwise = [ [dotProd row col | col <- m2Transposed ] | row <- m1 ]
53    where
54      cols1 = length (head m1)
55      rows2 = length m2
56      dotProd v1 v2 = sum (zipWith (*) v1 v2)
57      m2Transposed = rotateLList m2
58
59  binary2Bipolar :: (Ord a,Num a) => SimpMat a -> SimpMat a
60  binary2Bipolar inmat =
61    [[ if x <= 0 then -1 else 1  | x <- row ]  | row <- inmat ]
62
63  zeroDiagonal :: Num a => [[a]] -> [[a]]
64  zeroDiagonal  = zipWith zeroDiagRow [0..]
65    where
66      zeroDiagRow i  = zipWith (\j x -> if i == j then 0 else x) [0..]
67
68  scalarMultiply :: Num a => a -> SimpMat a -> SimpMat a
69  scalarMultiply eta inmat =
70    [[ eta * j | j <- row ] | row <- inmat]

import MyMats
putStrLn $ "a = "
putStr $ prettyMat mata
putStrLn $ "\nb = "
putStr $ prettyMat matb
putStrLn $ "\na + b = "
putStr $ prettyMat $ add2SimpMats mata matb
```

Figure 8: Adding Two Matrices

```
a =
[1,2,3]
[4,5,6]
b =
[-1,-2,-3]
[4,5,6]
a + b =
```

```
[0,0,0]
[8,10,12]

putStrLn $ "c = "
putStr $ prettyMat matc
putStrLn $ "\nd = "
putStr $ prettyMat matd
putStrLn $ "\nc * d ="
putStr $ prettyMat $ multSimpMats matc matd
```

Figure 9: Multiplying Two Matrices

```
c =
[1,2]
[3,4]
[5,6]
d =
[7,8,9]
[10,11,12]
c * d =
[27,30,33]
[61,68,75]
[95,106,117]
```

Classroom Exercise

Can you give the answer for

$$d \cdot c$$

?

Classroom Exercise

Can you demonstrate that matrix multiplication is not commutative?

2. Thinking Abstractly There are in fact many ways to think about what a vector is. It can be thought of as a column (or row of numbers). More abstractly it is an object (arrow) with magnitude and direction. Most abstractly it is anything that obeys the requirements of a vector space.

For particular circumstances one or another of the different definitions may serve our purposes better. In application to neural networks we often just use the first definition, a column of numbers, but the second can be more helpful for developing our geometric intuitions about what various learning rules are doing and how they do it.

Similarly, we may consider a matrix as a collection of vectors or it may be more convenient to think of it as a rectangular array of numbers.

Classroom Exercise

> What is the library or packge in the programming language that you will use for this course that has vectors and matrix functions?

3. Common Notational Conventions for Vectors and Matrices

   Vectors tend to be notated as *lower case* letters, often in bold, such as $\mathbf{a}$. They are also occasionally represented with little arrows on top such as $\overrightarrow{\mathbf{a}}$.

   Matrices tend to be notated as *upper case* letters, typically in bold, such as $\mathbf{M}$.

   Good things to know: what is an inner product? How does it differ from a scalar product. Can you write code in your preferred language for the inner product?

#### 4.1.5 Neural Networks Redux

#### 4.1.6 What is a Neural Network?

What is a Neural Network? It is a brain inspired computational approach in which "neurons" compute functions of their inputs and pass on a *weighted* proportion to the next neuron in the chain.

1. Non-linearities The spiking of a biological neuron is non-linear. You will see this in both the integrate and fire and Hodgkin and Huxley models you're going to program. For our neural networks we will usually use a *threshold*. When the threshold exceeds a certain value our activity will jump (or step) to a new value.

$$\text{if } I_1 \times w_{1,1} + I_2 \times w_{2,1} + I_3 \times w_{3,1} > \Theta \text{ then } Output = 1 \qquad (9)$$

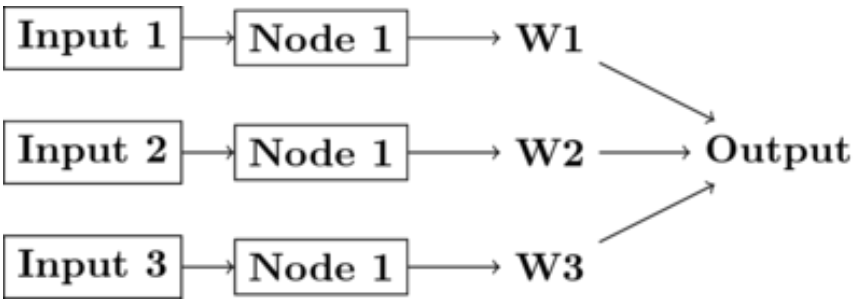



Figure 10: Simple schematic of the basics of a neural network. This is an image for a single neuron. The input has three elements and each of these connects to the same neuron ("node 1"). The activity at those nodes is filtered by the weights, which are specific for each of the inputs. These three processed inputs are combined to generate the output from this neuron. For multiple layers this output becomes an input for the next neuron along the chain.

What this equation shows is that Inputs ("I"'s) are passed to a neuron (also called a node or sometimes unit). Those inputs have something like a synapse. That is designated by the w's for weights. Those weights are how tightly the input and internal activity of our artificial neuron are coupled. The reason for all the subscripts is to try and help you see the similarity between this equation and the inner product and matrix multiplication rules you just worked on programming. The activity of the neuron is a sort of internal state, and then, based on the comparison of that activity to the threshold, you can envision the neuron spiking or not, meaning it has value 1 or 0. Mathematically, the weighted sum is fed into a threshold function that compares the value to a threshold ( $\Theta$ ), and passes on the value 1 if it is greater than the threshold and 0 (sometimes -1) for the inactive state.

To prepare you for the next steps in writing a simple perceptron (the earliest form of artificial neural network), you should try to answer the following questions.

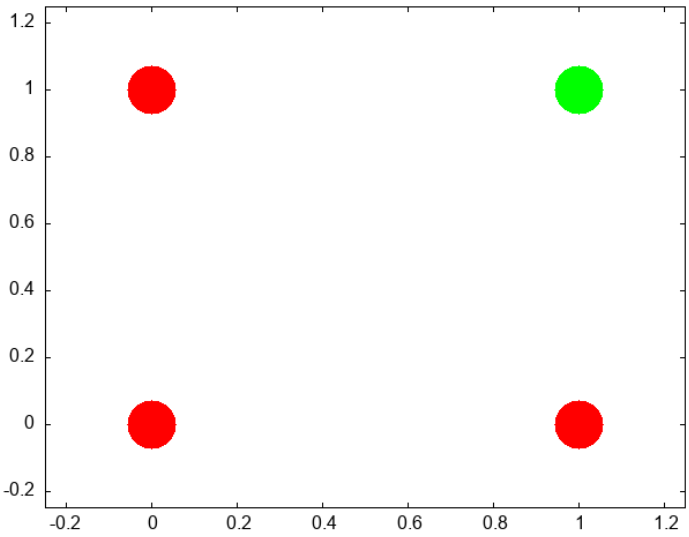
Classroom Questions

- What, geometrically speaking, is a plane?
- What is a hyperplane?
- What is linearly separability and how does that relate to planes and hyperplanes?

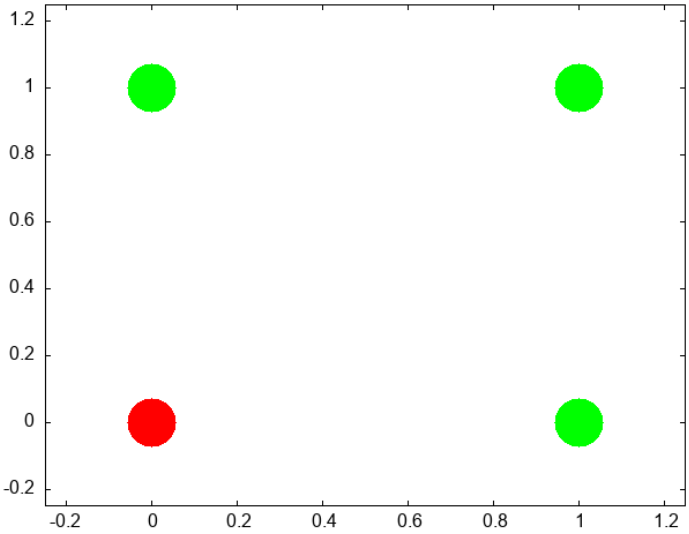
One of our first efforts will be to code a *perceptron* to solve the XOR problem. In order for this to happen you should know a bit about *Boolean* functions and what an XOR problem actually is.

(a) Examples of Boolean Functions and How They Map onto our Neural Network Intuitions

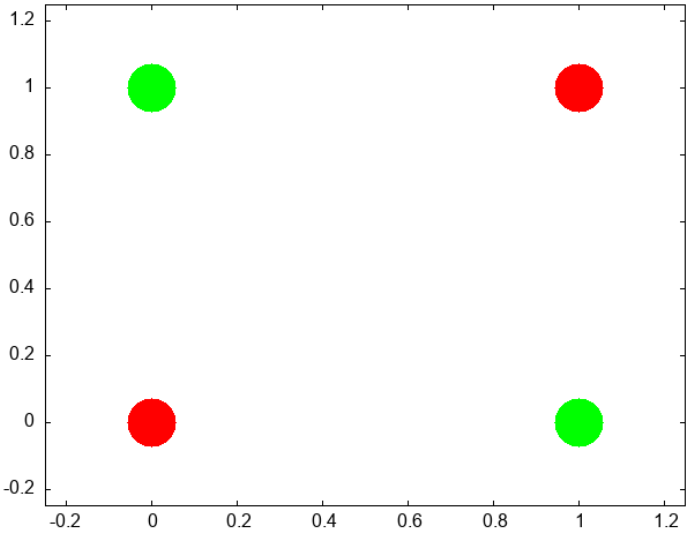
**The "AND" Operation/Function**




**The "OR" Operation/Function**




**The "XOR" Operation/Function**




Can you draw the truth tables that match each of these graphs?

This short article provides a nice example of linear separability and some basics of what a neural network is.

2. Exercise XOR

Using only **not**, **and**, and **or** operations draw the diagram that allows you to compute in two steps the **xor** operation. You will need this logic to code up a perceptron.

3. Connections Can neural networks encode logic? Is the processing zeros and ones enough to capture the richness of human intellectual activity?

   There is a long tradition of representing human thought as the consequence of some sort of calculation of two values (true or false). If you have two values you can swap out 1's and 0's for the true and false in your calculation. They even seem to obey similar laws. The conjunction (AND) of two true things is only true when both are true. If you take T = 1, then T AND T is the same as $1 \times 1$.

   We will next build up a simple threshold neural unit and try to calculate some of these truth functions with our neuron. We will build simple neurons for truth tables (like those that follow), and string them together into an argument. Then we can feed values of T and F into our network and let it calculate the XOR problem.

4. Boolean Logic George Boole, Author of the *Laws of Thought*.

   `https://archive.org/details/investigationofl00boolrich` `https://plato.stanford.edu/entries/boole/#LifWor`

5. First Order Logic - Truth Tables An example **Or**

| First | Second | Truth |
|---|---|---|
| 0 | 0 | FALSE |
| 0 | 1 | TRUE |
| 1 | 0 | TRUE |
| 1 | 1 | TRUE |

### 4.1.7 Our First Neural Network: The Perceptron

In many ways the Perceptron can be regarded as the first neural network ever.

The perceptron (see Figure 11) was the invention of a psychologist, Frank Rosenblatt (pdf). He was not a computer scientist. Though he obviously had a bit of the mathematician in him.

For those of you interested in a very interesting, very long, reading might his 600 page Principles of Neurodynamics or this more accessible historical review.

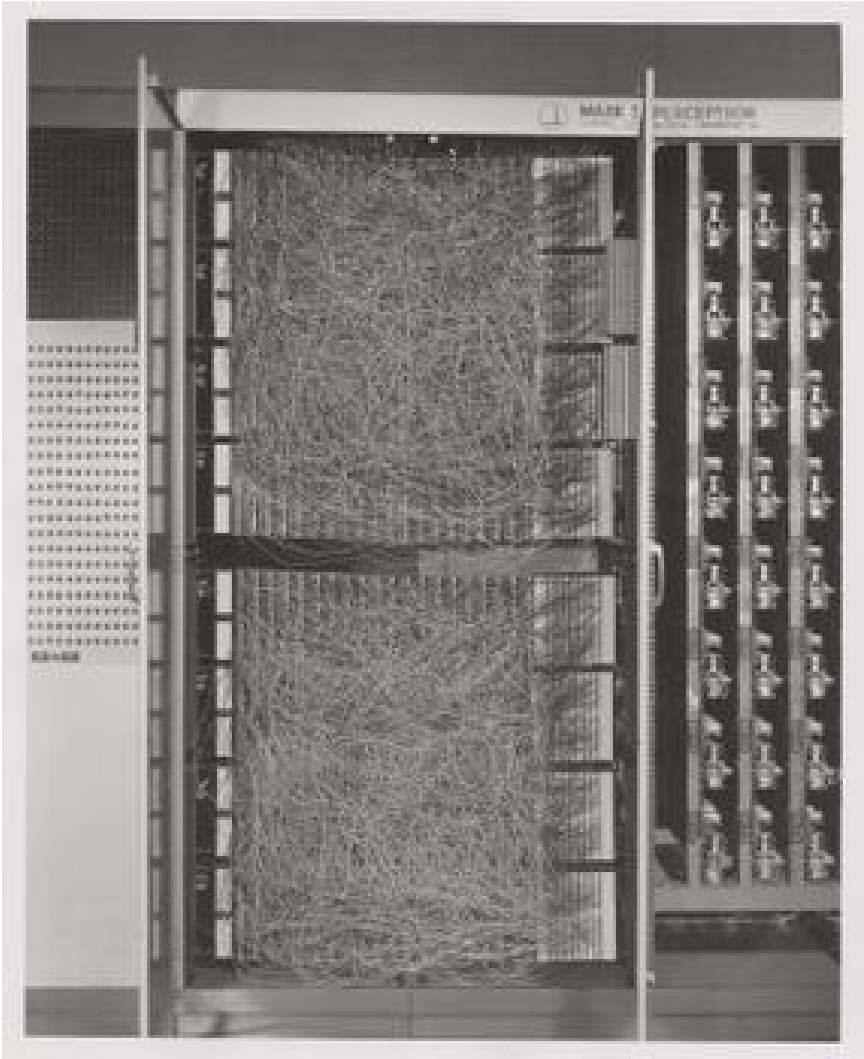


Figure 11: The Mark I computer for computing the perceptron's activity. Details can be found on Wikipedia's page.

But a good intuition for Rosenblatt's frame of mind with this research can be gained just from the foreword:

> For this writer, the perceptron program is not primarily concerned with the invention of devices for "artificial intelligence", but rather with investigating the physical structures and neurodynamic principles which under lie "natural intelligence". A perceptron is first and fore most a brain model, not an invention for pattern recognition. As a brain model, its utility is in enabling us to determine the physical conditions for the emergence of various psychological properties.

1. The Perceptron Rules The algorithm the perceptron follows is simple enough to be done by hand. But before doing that take a minute to think what is trying to do. It is trying to go from one set of "weights" to another set of "weights". If you think of all the weights as coordinates in a Cartesian graph then you can visualize each cycle of updating as adding another point to the graph. Squint your eyes and the dots merge to a path. A path through the weight *space.* This is a dynamic process. A theme that we return to in a later Chapter (add link here).

   In short, the perceptron "rules" are the equations that characterize what a perceptron should do when it gets some input, computes and output and learns if its guess is right or wrong. It revises its behavior based on this feedback, and thus can be said to learn. A lot can be done with these simple equations.

   $I = \sum_{i=1}^{n} w_i \; x_i$

   If $I \geq T$ then $y = +1$ else if $I < T$ then $y = -1$

   If the answer was correct, then $\beta = +1$, else if the answer was incorrect then $\beta = -1$.

   The "T" in the above equation refers to the threshold. This is a user defined value that is usually set to zero for convenience.

   Updating is done by $\mathbf{w_{new}} = \mathbf{w_{old}} + \beta y \mathbf{x}$

2. Be The Perceptron

   This is a pencil and paper exercise. Before coding it is often a good idea to try and work the basics out by hand. This may be a flow chart or a simple hand worked example. This both gives you a simple test case to compare your code against, but more importantly makes sure

that you understand what you are trying to code. Let's make sure you understand how to compute the perceptron learning rule, but doing a simple case by hand.

Beginning with an input of 0.3 0.7

And an initial set of weights of -0.6 0.8

and a class of 1.

Compute the value of the new weight vector with pen and paper.

```
updWeight0 ([[0.3,0.7]],1) [[-0.6,0.8::Double]]
```

```
[[-0.3,1.5]]
```

**Next steps.** Consider this as the data matrix where the first two elements in a row are the two inputs and the third element in a row is the "class" of the output.

$$\begin{bmatrix} 0.3 & 0.7 & 1.0 \\ -0.5 & 0.3 & -1.0 \\ 0.7 & 0.3 & 1.0 \\ -0.2 & -0.8 & -1.0 \end{bmatrix}$$

using the starting weight -0.6 0.8 try a few cycles through the data by hand and then move on to writing the perceptron rule into code so that you can automate the updating step. Haskell code for some of these functions can be found here.

```
:set -Wno-name-shadowing
putStrLn $ unlines $ map show testData
putStr "\n\n\n"
newwt = last $ oneRound myw testDataFs
checkWt newwt testData
putStr "\n\n"
newwt2 = last $ oneRound newwt testDataFs
checkWt newwt2 testData
```

```
([[0.3,0.7]],1.0)
([[-0.5,0.3]],-1.0)
```

```
([[0.7,0.3]],1.0)
([[-0.2,-0.8]],-1.0)

[(1.0,1.0),(1.0,-1.0),(1.0,1.0),(-1.0,-1.0)]

[(1.0,1.0),(-1.0,-1.0),(1.0,1.0),(-1.0,-1.0)]
```

(a) More Abstract Thinking Each time through the perceptron rule new weights are computed. Imagine these two weights as coordinates for the head of a vector with its base at the origin of a Cartesian plane. Imagine as you iterate that you are rotating your weight vector around an origin.

The decision plane for your network is perpendicular to the vectors and is anchored at the bottom of the arrow. If you could compare this plane to the location of the data points you would see that the rule is learning to find the decision plane that puts all of one class on one side of the line and all of the other class on the other side. That is why it is limited to problems that are linearly separable.

i. Bias is a Good Thing (at least here it is)
These data were selected such that the base of the vector could separate them while anchored at zero. However, for many data sets you not only need to learn what direction to point the vector, but you also need to learn where to anchor the vector. This is done by including a @bold{bias weight}. Add an extra dimension to your weight vector and your inputs. For the inputs it will just be a constant value of 1.0, but this extra, bias weight, will also be learned and allows you to achieve, effectively, a translation away from the origin to be able to separate points that are more heterogeneously scattered.

ii. Geometrical Thinking
- What is the relation between the inner product of two vectors and the cosine of the angle between them?
- What is the **sign** for the cosine of angles less than 90 degrees and those greater than 90 degrees?
- How do these facts help us to answer the question above?
- Why does this reinforce the advice to think *geometrically* when thinking about networks and weight vectors?

### 4.1.8 The Delta Rule - Homework

The **Delta Rule** is another simple learning rule that is a minimal variation on the perceptron rule. It is used more frequently, and it has the spirit of Hebbian learning, which we will learn more about soon. The homework asks you to write code to test and train an artificial neuron using the delta learning rule.

- For an easy start create some pseudo random linearly separable points on a a sheet of paper. Label one population as **1** and the other population as **-1**.
- For a more challenging set-up create the data programmatically using random numbers and some method that allows you to vary how close or distant the points are to the line of separation, and how many points there are to train on.
- The Delta Learning rule is: $\Delta\ w_i = x_i\ \eta(desired - observed)$
- Submit your code that has your test data in it. Start with an initial random weight and use the delta rule to learn the correct weighting to solve all your training examples. Then test on a new set of points that you did not test on but that are classified according to the same rule. Your code should assess how well the trained rule classifies the test data.

### 4.1.9 Not all Networks are the Same: Hopfield

- Feedforward
- Recurrent
- Convolutional
- Multilevel
- Supervised
- Unsupervised

The Hopfield network[4] has taught many lessons, both practical and conceptual. Hopfield showed physicists a new realm for their skills and added

[4] J J Hopfield, "Neural Networks and Physical Systems with Emergent Collective Computational Abilities.," *Proceedings of the National Academy of Sciences* 79, no. 8 (April 1982): 2554–58, https://doi.org/10.1073/pnas.79.8.2554.

recurrent (i.e. feedback) connections to network design (output becomes input). He changed the focus from network architecture to that of a dynamical system. Hopfield showed that the network could remember and it could do some error correction, it could reconstruct the "right" answer from faulty input.

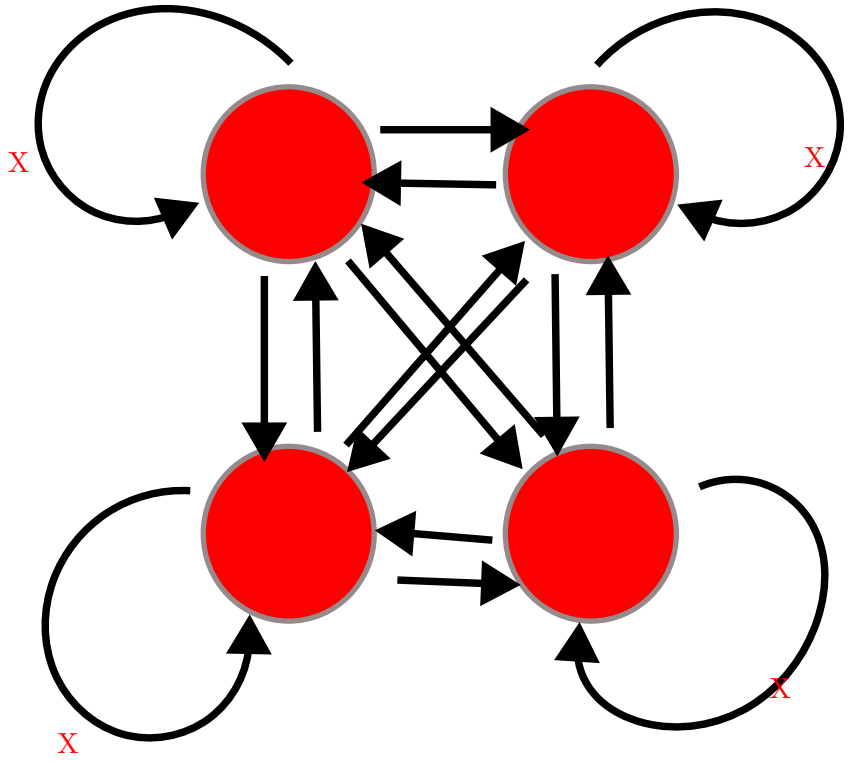



Figure 12: Illustration of a Hopfield Network's Connection Pattern With Four Nodes

1. How a Hopfield Network Operates

    - Each node has a value.
    - Each of those arrowheads has an associated weight.
    - The line with the "x" indicates that there are no self connections.
    - All other connections for all other units are present and go in both directions, and are typically the same in both directions.

    **Test Your Understanding**

- What does the input for a network like this with four nodes look like when framed in the linear algebra terms we just learned?
- A weight is a number associated to each connection. Tell me what the weights should look like in terms of the linear algebra constructs.
- How might we conceive of "running" the network for one cycle in terms of the above?

2. A Worked Example

Inputs can be thought of as vectors. Although I have drawn the network like a square that shape is really independent of the structure of data flow. Each node needs an input and each node will need a weighted contact to all the other nodes. Consider the following two input patterns and the following weight matrix.

$\overrightarrow{a} = [1, 0, 1, 0]^T$

$\overrightarrow{b} = [0, 1, 0, 1]^T$

$$weights = \begin{bmatrix} 0 & 3 & -3 & 3 \\ 3 & 0 & 3 & -3 \\ -3 & 3 & 0 & 3 \\ 3 & -3 & 3 & 0 \end{bmatrix}$$

How do you compute the output? Which comes first: the matrix or the input vector and why?

```
hopa = [[1,0,1,0]] :: SimpMat Integer
hopb = [[0,1,0,1]] :: SimpMat Integer
outProda = multSimpMats (binary2Bipolar (rotateLList hopa)) $ binary2Bipolar hop
outProdb = multSimpMats (binary2Bipolar (rotateLList hopb)) $ binary2Bipolar hop
outProdBoth = add2SimpMats outProda outProdb
noSelfConnections  = zeroDiagonal outProdBoth
learnRate = scalarMultiply 1 noSelfConnections
putStr $ prettyMat learnRate
```

```
[0,-2,2,-2]
[-2,0,-2,2]
[2,-2,0,-2]
[-2,2,-2,0]
```

Hopfield networks use a threshold rule. This non-linearity is, at least metaphorically, like the threshold that says whether a neuron in the brain or in our integrate and fire model fires. For the Hopfield network our threshold rule says:

$$output(t) = \begin{array}{l} 1 \text{ if } t \geq \Theta \\ 0 \text{ if } t < \Theta \end{array}$$

$\Theta$ represents the value of our threshold and for now set it to zero.

**In Class Activity**

- Calculate the output to each of the two input patterns.
- Then to test your intuition, you should guess what output you would get for an input of $[1, 0, 0, 0]^T$.
- Calculate it.
- Is A or B is **closer** to this test input?
- What does it means, in this context, for one vector to be "closer" to another?

(a) An Aside Distance Metrics

Metrics relate to measurement. For some operation to be a distance metric it should meet three intuitive requirements and one that is maybe not as obvious. To measure the distance between two things we need an operation that is binary. That is, it takes two inputs. In this case that would be our two vectors. It's result should always be **non-negative**. A negative distance would clearly be meaningless. Our output should be **symmetric**. Meaning that $d(A, B) = d(B, A)$. The distance from Waterloo to Toronto ought to come out as the same as going from Toronto to Waterloo. Our metric should be **reflexive**. The distance from anything to itself ought to be zero. Lastly, to be a distance metric, our operation must obey the triangle inequality.

The familiar distance measure of everyday life is the Euclidean. A distance measure calculated as the number of mismatched bits is the *Hamming* distance.

For perceptrons we talked about how the weight vector moved the direction it pointed. Here we don't have the weight vector moving, but you can visualize what is happening as updating a point in space. When we first input our four element vector we have a location in 4-D space. We multiply the first row of our

weight matrix against our column of the input vector and we see, in effect, what is the effect on our first element (node) of all the other weighted inputs coming in to it. We then "update" that location. Maybe we flip it from a 1 to a zero (or vice versa). Then we try the next row of the weight matrix to see what happens to the second element. As we change the values of our nodes we are creating new points. The sequence of points is a trajectory that we are tracing in the input space. In this simple situation here we only require one pass to reach the final location, but in other settings we might not. In that case we just keep repeating the process until we do. One of the wonderful insights that Hopfield had was that by conceptualizing this process as an "energy" he could mathematically prove that the process would always reach a resting place.

3. Hebb's Outer Product Rule

   **Ask yourself**

   Why is this learning rule called "Hebb's"? Who was the Hebb it is named after?

   The strength of a change in a connection is proportionate to the product of the input and outputs, i.e. $\Delta A[i,j] = \eta f[j] g[i]$ and $g[i] = \sum_j A[i,j]\ f[j]$ therefore, $\vec{g} = \mathbf{Af}$.[5]

   **You will want to know**

   - What is an outer product?
   - How do you compute it in your programming language of choice?

#### 4.1.10 Hopfield Homework Description: Robustness to Noise (this might take awhile).

Overview of the steps to take:

- Create a small set of random data for input patterns.
- Generate the weights necessary to properly decode the inputs.

[5] Does it matter that the **W** comes first?

- Conceive of a way to randomly corrupt the inputs. Perhaps by flipping some bits and show that your network does correctly decode the uncorrupted inputs.
- Report the accuracy of the output. Explore how the length of the input vector and the number of bits your "flip" impact performance.

More detailed instructions:

- Make the input patterns 2-d, square and of size "n".
- Use a bipolar system and have, roughly, equal numbers of +1s and -1s in your patterns.
- Make a few of them and store them in some sort of data structure.
- Using those patterns, compute the weight matrix with the following equation: $w_{ij} = \frac{1}{N} \sum_{\mu} value_i^{\mu} \times value_j^{\mu}$ Where N is the size of the patterns, that is how many "neurons". $\mu$ is an index for each of the patterns, and $i$ and $j$ refer to the neurons in the pattern.
- Program an **asynchronous** updating rule, run your network until it stabilizes, and then show that you get back what you put in.
- Then do the same for at least one disrupted pattern (where you flipped a couple of bits around.)

# 5 Theory Creation in the Cognitive and Psychological Sciences

> I have been wondering about the concept of explanation. I don't know if there is an agreed upon definition in this context. How do we know that an observation is "explained" other than that it is predicted by a theory? Or is the mathematics the same, and is explanation synonymous with post-diction? – Joachim Vanderkerckhove

## 5.1 What is a scientific theory?

And how do the notions of prediction and explanation fit into this? Is the ability to predict the tides a theory of the tides? Is it an explanation of the tides?

### 5.1.1 What is prediction?

And what makes for a good/bad prediction?

### 5.1.2 What is explanation?

And, again, good v. bad explanations?

## 5.2 Classroom Activity

Pick anything you think is an example of a good scientific theory. Be prepared to name it and explain it and to respond to questions that critique it.

## 5.3 Do we have a theory crisis in Psychology?[6]

You might find these citations relevant for discussion.[7]

## 5.4 Classroom Activity

Did you pick a psychological example? Do you agree that there is a theory crisis in Psychology?

---

[6] Klaus Oberauer and Stephan Lewandowsky, "Addressing the Theory Crisis in Psychology," *Psychonomic Bulletin & Review* 26, no. 5 (2019): 1596–1618.

[7] Olivia Guest and Andrea E. Martin, "How Computational Modeling Can Force Theory Building in Psychological Science," *Perspectives on Psychological Science*, 2021, 174569162097058, `https://doi.org/10.1177/1745691620970585`; Iris van Rooij and Giosuè Baggio, "Theory before the Test: How to Build High-Verisimilitude Explanatory Theories in Psychological Science," *Perspectives on Psychological Science*, 2021, 174569162097060, `https://doi.org/10.1177/1745691620970604`; Denny Borsboom et al., "Theory Construction Methodology: A Practical Framework for Building Theories in Psychology," *Perspectives on Psychological Science* 16, no. 4 (February 2021): 756–66, `https://doi.org/10.1177/1745691620969647`; Peter Naur, "Programming as Theory Building," *Microprocessing and Microprogramming* 15, no. 5 (1985): 253–61; Markus I. Eronen and Jan-Willem Romeijn, "Philosophy of Science and the Formalization of Psychological Theory," *Theory &Amp; Psychology* 30, no. 6 (December 2020): 786–99, `https://doi.org/10.1177/0959354320969876`; Tobias Koch and Alexander Wendt, "Theory Crises in Psychology – What Can We Learn from Structuralism?," April 2025, `https://doi.org/10.31219/osf.io/y7xbz_v1`; Freek Oude Maatman, "Psychology's Theory Crisis, and Why Formal Modelling Cannot Solve It," July 2021, `https://doi.org/10.31234/osf.io/puqvs`; David M. Sanbonmatsu, Becky Neufeld, and Steven S. Posavac, "There Is No Theory Crisis in Psychological Science.," *Journal of Theoretical and Philosophical Psychology*, January 2025, `https://doi.org/10.1037/teo0000301`.

## 5.5 What are scientific theories good for? Or, in other words, why should we care?

## 5.6 What makes a theory a good theory?

Try to answer this by using positive and negative examples from the theories selected by your classmates.

## 5.7 What are the first steps towards building a theory?

- Phenomena
- Analogies
- Should we adhere to a deductive-nomological model?
- Do we want a *causal* model? What do we mean, really, by A causes B and how can we assess it?

## 5.8 What should you do next if you think you have a theory?

### 5.8.1 What role should a model play?

1. Can a model be verbal?
2. Is it enough to express a model in code or pseudo-code?
3. Is mathematics necessary?

## 5.9 Formal Modeling (Advocacy)

### 5.9.1 Advantages

1. Everything must be defined
2. The importance of actual values cannot be avoided
3. Inconsistencies and gaps made obvious
4. Can be run on a computer? Is this really an advantage? What are the guarantees that the code we run is a faithful implementation of our ideas? It is interesting to me that the example used in the notes I am borrowing from use Einstein and Eddington's experience with solar eclipses, but this did not use a computer program. Instead it was just math.

## 5.10 Steps to Formalizing a Model

- Define the phenomena (not the specific experiments, but the phenomena writ large [the larger the better]).
- Are there levels?
- Can you specify processes?
- Be open to emergence. Do the automata we programmed show emergence. We only specified the colors of pixels, but patterns appeared.
- Can you find a mathematical form for your process (e.g. exponential)?

## 5.11 Can you now provide a good example of a psychological theory?

## 5.12 Some Interesting Reading on this Topic

(In addition to the references listed below).

1. Formalizing Verbal Theories. A tutorial by dialogue.
2. Forms of explanation and understanding for neuroscience and artificial intelligence
3. Models are Stupid, and We Need More of Them
4. Productive Explanation: A framework for evaluating explanations in psychological science
5. Theory Construction Methodology: A practical framework for building theories in psychology

## 5.13 Theory Homework

In "Models are Stupid"(see link above) Smaldino writes:

**Hopfield's Model of Content-Addressable Memory**

Memory - the ability to store information for later recall—is a wondrous property of neural networks that makes possible all but the most rudimentary forms of cognition. By the early 1980s, long-term potentiation—the process by which Donald Hebb's theory that "neurons that fire together wire together" occurs—was relatively well described. It was believed that the formation of such associations was intrinsic to more complex forms of memory,

such as that by which a person's face is encoded and then later recognized, but the mechanism was unclear. How could a brain possibly use partial information, like an occluded face, to reconstruct information encoded in memory? To begin to answer this question, the biophysicist John Hopfield (1982) constructed a simple model of two-state neurons in a fully connected network. Edge weights were determined by a process of Hebbian learning assumed to have already occurred, so that a number of configurations(or states) of "on" and "off " neurons were encoded in the network, with edges assumed to be bidirectionally symmetrical (i.e., undirected). Using mathematics derived from statistical physics, Hopfield showed firstly that in such a system, encoded states would be stable, and secondly that if initialized in a non-encoded state, the network would self-organize into the encoded state that most closely matched the initialized state. In other words, he had a model for how memory retrieval could emerge spontaneously in a simple neural network. This model is almost absurdly simplistic, even stupid in its assumptions. Neurons are either on or off, ignoring subtleties of firing rates or even graded activation. Directionality is also ignored; links between neurons are equally strong in each direction. Exactly how the network is presumed to first arrive in its initial state is left a mystery. Yet analysis of the model showed that something like biological neural networks could produce content-addressable memory. Hopfield himself later showed that the model's functioning was robust to the relaxation of some of his strict assumptions (Hopfield, 1984), and the work has laid the foundation for much subsequent work in understanding the neurobiology of memory.

Your task is to answer the question: Does Hopfield express a theory? Make sure you address the issues of what constitutes a theory that we discussed above and that you include any relevant facts from your own Hopfield model code and experiments.

Is this a theory of memory? Are there room for others? What is missing, if anything?

This is an opportunity for you to think deeply and critically about theory. A good foundation now will make you a wiser, probably more skeptical, consumer of future research so put your back into it and let me see some of your best work. Write clearly, logically, and persuasively. I will leave length up to you. I am not looking for how many words you write, but the completeness of your ideas in the development and defense of your thesis.

# 6 Algebraic Psychology

This section, still to be written, will present some of the mathematics of abstract algebra (not as hard as it sounds), and then show how that can be used to approach a cognitive modeling problem. The example problem we will use is a comparison of *formal concept analysis* with Paul Thagard's *Theory of Explanatory Coherence*.[8]

As a placeholder, and in case I run out of time, I will insert some of the slides I will be using March 12, 2026 for a lecture in CogSci 600.

## 6.1 Opening Questions

### 6.1.1 What is a theory?

### 6.1.2 Name a psychological theory.

## 6.2 I demand a satisfactory explanation

### 6.2.1 A celestial example and a Planetary Context

| | size | | | distance from sun | | moon | |
|---|---|---|---|---|---|---|---|
| | small | medium | large | near | far | yes | no |
| **Mercury** | × | | | × | | | × |
| **Venus** | × | | | × | | | × |
| **Earth** | × | | | × | | × | |
| **Mars** | × | | | × | | × | |
| **Jupiter** | | | × | | × | × | |
| **Saturn** | | | × | | × | × | |
| **Uranus** | | × | | | × | × | |
| **Neptune** | | × | | | × | × | |
| **Pluto** | × | | | | × | × | |

## 6.3 Thagard's Principles of Explanatory Coherence

1. Symmetry
2. Analogy
3. Contradiction
4. Competition
5. Acceptability

[8] Paul Thagard, "Explanatory Coherence," *Behavioral and Brain Sciences* 12, no. 3 (September 1989): 435–67, https://doi.org/10.1017/s0140525x00057046.

6. Data Priority

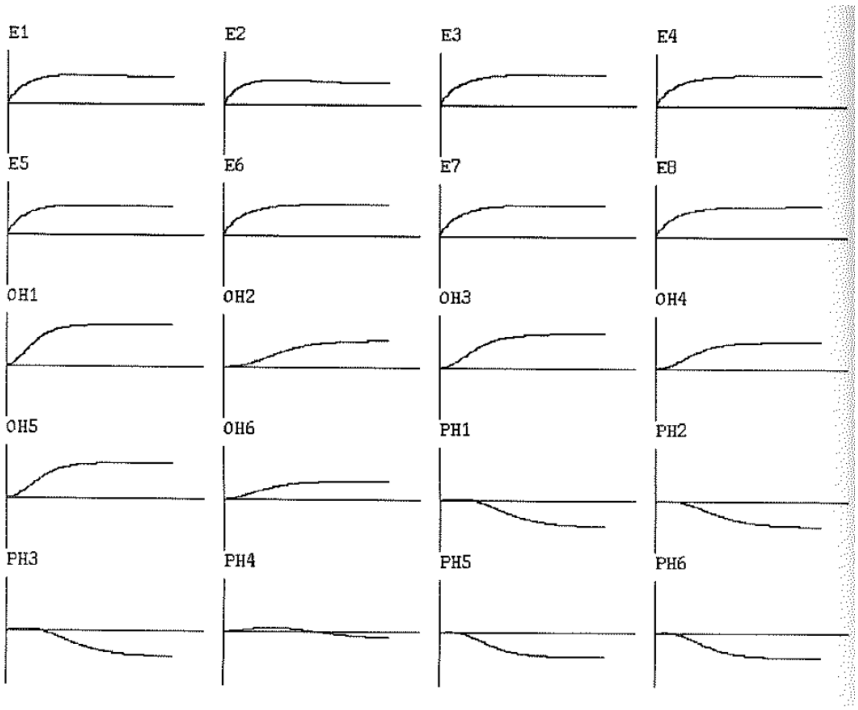
7. Explanation

## 6.4 Taking up all the oxygen in the room

Table I. *Input propositions for Lavoisier (1862) example*

| | | |
|---|---|---|
| *Evidence* | | |
| (proposition | 'E1 | "In combustion, heat and light are given off.") |
| (proposition | 'E2 | "Inflammability is transmittable from one body to another.") |
| (proposition | 'E3 | "Combusion only occurs in the presence of pure air.") |
| (proposition | 'E4 | "Increase in weight of a burned body is exactly equal to weight of air absorbed.") |
| (proposition | 'E5 | "Metals undergo calcination.") |
| (proposition | 'E6 | "In calcination, bodies increase weight.") |
| (proposition | 'E7 | "In calcination, volume of air diminishes.") |
| (proposition | 'E8 | "In reduction, effervescence appears.") |
| *Oxygen hypotheses* | | |
| (proposition | 'OH1 | "Pore air contains oxygen principle.") |
| (proposition | 'OH2 | "Pure air contains matter of fire and heat.") |
| (proposition | 'OH3 | "In combustion, oxygen from the air combines with the burning body.") |
| (proposition | 'OH4 | "Oxygen has weight.") |
| (proposition | 'OH5 | "In calcination, metals add oxygen to become calxes.") |
| (proposition | 'OH6 | "In reduction, oxygen is given off.") |
| *Phlogiston hypotheses* | | |
| (proposition | 'PH1 | "Combustible bodies contain phlogiston.") |
| (proposition | 'PH2 | "Combustible bodies contain matter of heat.") |
| (proposition | 'PH3 | "In combustion, phlogiston is given off.") |
| (proposition | 'PH4 | "Phlogiston can pass from one body to another.") |
| (proposition | 'PH5 | "Metals contain phlogiston.") |
| (proposition | 'PH6 | "In calcination, phlogiston is given off.") |

## 6.5 And the network said ...

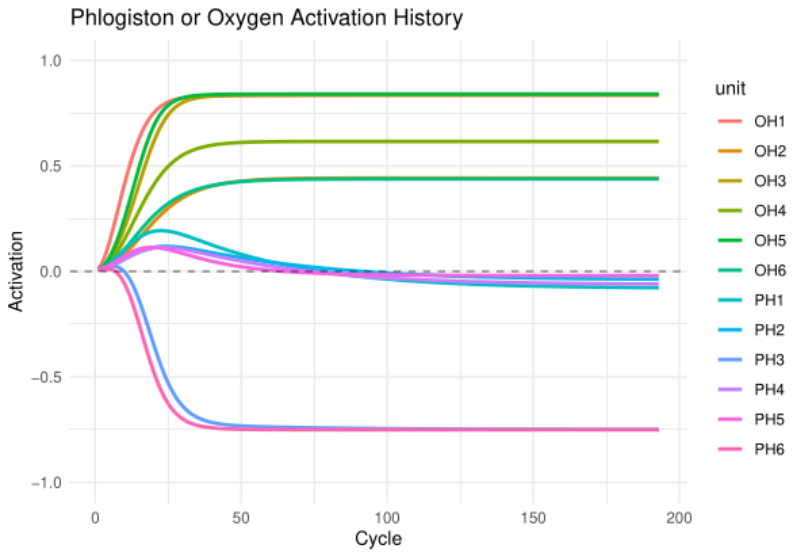



## 6.6 From Lisp to R

```
# ECHO Model in R
# Ported from Paul Thagard's MacLisp implementation
echo_env <- new.env()
echo_env$min_activation <- -1.0
echo_env$max_activation <- 1.0
create_unit <- function(name, activation = echo_env$default_activation) {
  unit <- list(
    name = name,
    activation = activation,
    new_activation = activation,
  )
weight_of_link_between <- function(unit1_name, unit2_name) {
  exp_unit <- get_unit(exp)
  exp_unit$explains <- unique(c(exp_unit$explains, explanandum))
}
# Run Lavoisier example
```

```
h2 <- example_lavoisier()
```

## 6.7 And the network said ... (ECHO, Echo, echo ...)




Can we do better?

## 6.8 Formal CONTEXTS

- A set of objects (we'll denote with $G$ by convention).
- A set of attributes (we'll denote with $M$ by convention).
- And a relation ($R$) between these two sets.
- It is often convenient to present this as a matrix (as we did for the planets).

## 6.9 Formal CONCEPTS

- Sets of objects that all share the same attributes.
- Sets of attributes that are all possesed by the same objects.
- Both with nothing left out.

- That is, for $A \subseteq G$ and $B \subseteq M$ if we define the following (') operations thusly:

$$A' = \{m \in M \mid \forall g \in A \; gRm\} \tag{10}$$
$$B' = \{g \in G \mid \forall m \in B \; gRm\} \tag{11}$$
$$\text{then, } A'' = A, \text{ defines a concept.} \tag{12}$$

## 6.10 A Romantic Connection

**Definition 1** (Galois Connection)**.** $a \leq F(b) \iff G(a) \geq b$

Where $a \subseteq A$, and $b \subseteq B$ with $F : B \to A$, and $G : A \to B$ bijective and monotonic functions.

Note that this works for our concepts.

## 6.11 A celestial example

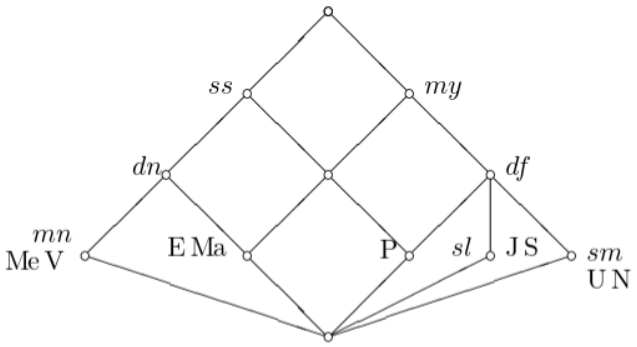
| | size | | | distance from sun | | moon | |
|---|---|---|---|---|---|---|---|
| | small | medium | large | near | far | yes | no |
| **Mercury** | × | | | × | | | × |
| **Venus** | × | | | × | | | × |
| **Earth** | × | | | × | | × | |
| **Mars** | × | | | × | | × | |
| **Jupiter** | | | × | | × | × | |
| **Saturn** | | | × | | × | × | |
| **Uranus** | | × | | | × | × | |
| **Neptune** | | × | | | × | × | |
| **Pluto** | × | | | | × | × | |

Is the combination of Earth and Mars a concept?

$$A = \{Earth,\ Mars\} \tag{13}$$
$$A' = \{small,\ near,\ moon\} \tag{14}$$
$$A'' = \{Earth,\ Mars\} \tag{15}$$

## 6.12 Jacob's Ladder

### 6.12.1 A Planetary Lattice




### 6.12.2 The Context

| | size | | | distance from sun | | moon | |
|---|---|---|---|---|---|---|---|
| | small | medium | large | near | far | yes | no |
| Mercury | × | | | × | | | × |
| Venus | × | | | × | | | × |
| Earth | × | | | × | | × | |
| Mars | × | | | × | | × | |
| Jupiter | | | × | | × | × | |
| Saturn | | | × | | × | × | |
| Uranus | | × | | | × | × | |
| Neptune | | × | | | × | × | |
| Pluto | × | | | | × | × | |

## 6.13 From ECHO to FCA: The Lavoisier Context

```
[1] "Lavoisier vs Phlogiston Explanatory Context:"
OH1 OH2 OH3 OH4 OH5 OH6 PH1 PH2 PH3 PH4 PH5 PH6
E1   1   1   1   0   0   0   1   1   1   0   0   0
E2   0   0   0   0   0   0   1   0   1   1   0   0
E3   1   0   1   0   0   0   0   0   0   0   0   0
E4   1   0   1   1   0   0   0   0   0   0   0   0
E5   1   0   0   0   1   0   0   0   0   0   1   1
E6   1   0   0   1   1   0   0   0   0   0   0   0
E7   1   0   0   0   1   0   0   0   0   0   0   0
E8   1   0   0   0   0   1   0   0   0   0   0   0
```

Evidence E1-E8 as objects, hypotheses OH1-OH6 (Oxygen) and PH1-PH6 (Phlogiston) as attributes. A value of 1 means the hypothesis explains the evidence.

### 6.14 Sample Formal Concepts

```
[[1]]
({E1, E2, E3, E4, E5, E6, E7, E8}, {})

[[2]]
({E1, E2}, {PH1, PH3})

[[3]]
({E2}, {PH1, PH3, PH4})

[[4]]
({E1, E3, E4, E5, E6, E7, E8}, {OH1})

[[5]]
({E8}, {OH1, OH6})
```

Each concept shows:

- **Extent**: Set of evidence explained together
- **Intent**: Set of hypotheses that explain exactly that evidence

### 6.15 Implications in the Lavoisier Context

```
[1] "Key implications discovered:"
Implication set with 3 implications.
Rule 1: {OH1, OH4, OH6} -> {OH2, OH3, OH5, PH1, PH2, PH3, PH4, PH5, PH6}
Rule 2: {OH1, OH4, OH5, PH5, PH6} -> {OH2, OH3, OH6, PH1, PH2, PH3, PH4}
Rule 3: {OH1, OH3, OH6} -> {OH2, OH4, OH5, PH1, PH2, PH3, PH4, PH5, PH6}
```

Implications reveal logical dependencies: if evidence supports certain hypotheses, what other hypotheses must also be supported?

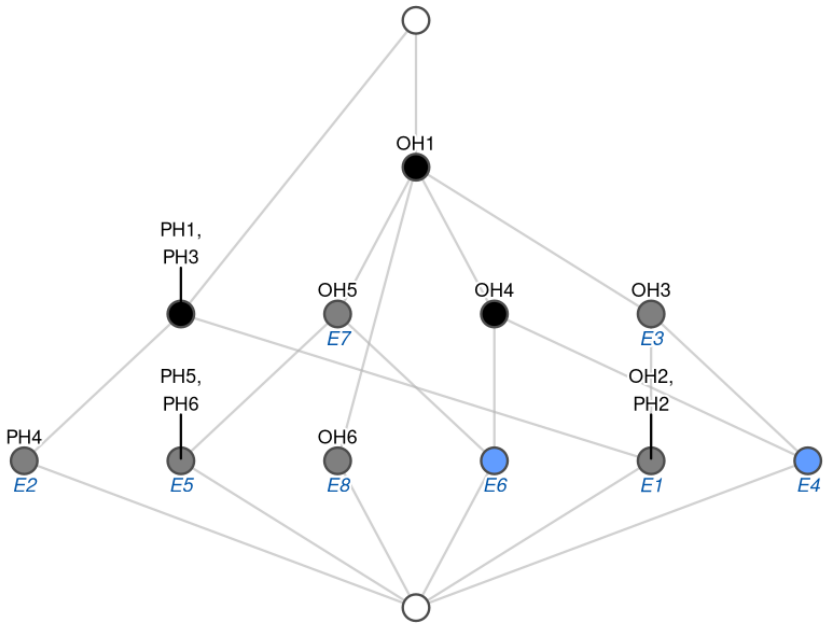



Figure 13: "The lattice structure reveals the hierarchical organization of explanatory relationships."

### 6.16 The Lavoisier Concept Lattice

# 7 Reasons to Prefer FCA

## 7.1 Data Processing Inequality

**Theorem 1** (Data Processing Inequality)**.** *For random variables $X$, $Y$, and $Z$ that form a Markov chain $X \to Y \to Z$, we have:*

$$I(X;Z) \leq I(X;Y)$$

*where $I(X;Y)$ denotes the mutual information between $X$ and $Y$.*

In our context: Information cannot be gained by processing data through additional theoretical layers.

## 7.2 Formal Contexts are Chu Spaces

**Definition 2** (Chu Space)**.** *A Chu space over a set $K$ is a triple $(A, r, X)$ where:*

- *$A$ and $X$ are sets (called* points *and* states *respectively)*
- *$r : A \times X \to K$ is a function (the* evaluation map*)*

*The Chu space defines a duality via:*

$$a^{\perp} = \{x \in X \mid r(a,x) = 0\}$$
$$x^{\perp} = \{a \in A \mid r(a,x) = 0\}$$

Formal contexts are Chu spaces over $\{0,1\}$ where objects are points, attributes are states, and the incidence relation is the evaluation map.

## 7.3 Chu Spaces are *-autonomous categories

In the category of Chu spaces, every object $A$ has a dual $A^*$ such that:

$$A^{**} \cong A$$

The *-autonomous structure provides:

- Duality: Each Chu space has a canonical dual
- Internal Hom: $[A, B] = A^* \otimes B$
- Self-duality: The category is equivalent to its own opposite

This categorical framework unifies logic, geometry, and computation through the lens of duality.

### 7.4 Unifying Logic, Geometry, and Computation

- Logic: Chu spaces model linear logic's multiplicative fragment, where $A \multimap B \equiv A^* \otimes B$
- Geometry: Stone duality connects topological spaces with Boolean algebras via Chu construction
- Computation: Game semantics emerges naturally—strategies are morphisms in Chu categories
- Cognition: Formal concepts capture the duality between extension (objects) and intension (properties)

## 8 But wait! There's more!

### 8.1 I probably ran out of time so come talk to me

- Monoids
- Monoidal Categories
- Monoidal and Comonoidal Objects
- Hypergraphs
- Categories of Hypergraphs
- Connections to Other Areas of Cognition

#### 8.1.1 Bibliography and Other Readings

## 9 Final Projects

(also available as a pdf on learn)

### 9.1 Overview

The goals of the final project are:

1. Active learning
2. Learning through teaching
3. Acquiring important professional skills such as:

- Presentation development
- Oral presentation practice
- Project Planning
- Group collaboration
- Technical Writing
- Coding Relevant to Psychology and Neuroscience Modeling
- Technical Knowledge

In pursuit of these goals a fair amount of time in the last few weeks will be saved for the group project. Each of you will have been assigned by me to a group. The members of the groups will work on the project collaboratively dividing the responsibilities and workload as jointly decided.

The deliverables will be two. First, there will be on the last course day a 1/2 hour presentation that will:

1. Provide a general introduction to the topic. This will not only be a technical introduction, but will help us understand the history and motivation for using this tool/technique.

2. A tutorial survey of the details. This might include the mathematical underpinnings and derivation as well as any pertinent novel notation.

3. Practical aspects of coding the algorithm. Ideally working code written by the group will be demonstrated. This is expected to be simple and modest in scope. So, it might be appropriate in addition to the group's own code to show off libraries or software written by others and useful for more complicated or realistic demonstrations.

The emphasis of the oral presentation is on learning. Both of us: the audience and the group presenting. This is a time to practice your presentation style and any novel ideas you have about how to give a memorable and instructive presentation. There will be no grade beyond a complete/incomplete.

The goal of the written reports is to provide an opportunity to develop and share a more rigorous and complete idea of the same general scope, but with increased detail and space for explanations. While a written document is expected other materials may make sense as part of the submission. For example, source code is likely to be a part of all projects. In addition, images or movies of simulations and similar materials may be appropriate. That is for the group to decide.

## 9.2 Evaluation

1. For the oral presentation the evaluation is strictly limited to completion. However, I hope that students will look upon this as an opportunity for practice, and also will be respectful of their peers time. You will be listening much more than speaking so keep that in mind as you plan the quality of your own offering.

2. For the written, final presentation I will be evaluating:

   - Clarity, completeness, and professionalism of the writing.
   - Documentation of the topic should include appropriate citations. I don't care so much about what citation style you use as long as it is consistent and follows some professional standard.
   - The tutorial code provided will be evaluated for style, accuracy, and the ease with which someone else (me for example) can get it working properly. That may mean the inclusion of instructions.
   - The value of any additional materials.
   - A general assessment of the pedagogical value of the presentation as a guide to help others understand the material. Ask yourself if your presentation would be a good guide for me as an instructor to use it as the basis for developing additional units of material to present in this course?

## 9.3 Project Topics

Each group will be assigned one of the topics below. Send me your groups ordered preference list as soon as possible. I have added some starter references, but those should not be viewed as sufficient in and of themselves. It is just to give a chance to get a better sense of what the topic is about and what kind of resources you might be looking for.

### 9.3.1 Reinforcement Learning

There is a "classic" textbook available on line, code as well. Reinforcement Learning is a common paradigm and mechanism for explaining a multitude of behaviors. The basic idea is that you do more of whatever it is that gives you a reward. The mapping of dopamine dynamics onto reward delivery has been heralded as one of the great successes of computational neuroscience.

1. Starting References and Resources

- The canonical textbook, freely available online, is Sutton & Barto's *Reinforcement Learning: An Introduction* (2nd ed.). Both the PDF and code examples are available at: `http://incompleteideas.net/book/the-book-2nd.html`
- For the neuroscience connection, see the dopamine and reward prediction error review cited below.
- A practical Python-focused tutorial series using OpenAI Gym (now Gymnasium) is a good hands-on complement: `https://gymnasium.farama.org/`

### 9.3.2 Self-organizing Maps (Kohonen Neural Networks)

Ever wonder how we develop those remarkable maps in the brain where there seems to be an organizational principle that puts similar things next to each other? There is an R package for Kohonen maps, and an article that describes it. For more on the idea itself there are many treatments. One possible starting point would be Kohonen himself.

1. Starting References and Resources

    - Kohonen, T. (1982). Self-organized formation of topologically correct feature maps. *Biological Cybernetics*, 43(1), 59–69. `https://doi.org/10.1007/BF00337288` — the original paper.
    - Wehrens, R., & Buydens, L. M. C. (2007). Self- and super-organizing maps in R: The kohonen package. *Journal of Statistical Software*, 21(5). `https://doi.org/10.18637/jss.v021.i05` — the primary reference for the R `kohonen` package.
    - The R `kohonen` package documentation and vignettes are a practical starting point for coding: `https://cran.r-project.org/package=kohonen`

### 9.3.3 Morris-Lecar Networks

In order to gain insights into the dynamics of models of neurons it is useful to simplify them from their fully biologically realistic instances. One simplified version is the Morris-Lecar model that has fewer moving parts, but yields rich dynamics. An introduction from a textbook is available on line. Code also can be found all over the internet, and you should have code, but this might also be a good chance to look at this tool widely used by the neural dynamics community. The author of the software also has his own textbook available for download.

1. Starting References and Resources

    - Morris, C., & Lecar, H. (1981). Voltage oscillations in the barnacle giant muscle fiber. *Biophysical Journal*, 35(1), 193–213. `https://doi.org/10.1016/S0006-3495(81)84782-0` — the original paper.
    - Rinzel, J., & Ermentrout, G. B. (1998). Analysis of neural excitability and oscillations. In C. Koch & I. Segev (Eds.), *Methods in Neuronal Modeling* (2nd ed., pp. 251–291). MIT Press. — a classic chapter situating Morris-Lecar in the broader landscape of neural dynamics. (I found an online copy here)
    - Ermentrout, B., & Terman, D. (2010). *Mathematical Foundations of Neuroscience*. Springer. — Ermentrout's textbook is freely downloadable and treats Morris-Lecar in depth; his simulation tool XPPAUT is also widely used: `http://www.math.pitt.edu/~bard/xpp/xpp.html` or `https://sites.pitt.edu/~phase/bard/bardware/xpp/xpp.html`.
    - The Brian2 simulator provides a Python environment well suited to implementing Morris-Lecar networks: `https://brian2.readthedocs.io/`

### 9.3.4 Backpropagation

This is the method that revolutionized the world of neural networks. It was a means of transmitting the error to each unit in a multilayer perceptron that would allow for effective quick learning of complex problems. The derivation of the method is complicated in the details, but relatively straightforward in the broad strokes. There are many, many demonstrations of this, but writing your own code to implement the method teaches you a lot.

1. Starting References and Resources

    - Rumelhart, D. E., Hinton, G. E., & Williams, R. J. (1986). Learning representations by back-propagating errors. *Nature*, 323, 533–536. `https://doi.org/10.1038/323533a0` — the landmark paper.
    - Nielsen, M. (2015). *Neural Networks and Deep Learning*. Free online book with worked examples and code in Python: `http://neuralnetworksanddeeplearning.com/` — an excellent and

readable tutorial treatment of backpropagation from first principles.

- 3Blue1Brown's video series "Neural Networks" on YouTube provides outstanding visual intuition for the algorithm before diving into the mathematics: `https://www.youtube.com/playlist?list=PLZHQObOWTQDNU6R1_67000Dx_ZCJB-3pi`

### 9.3.5 Agent Based Models

Agent Based Models (ABMs) allow us to simulate how complex, population-level patterns emerge from the local interactions of many individual agents, each following simple rules. They have been applied to problems ranging from the spread of disease to the emergence of social norms and collective neural dynamics.

1. Starting References and Resources

    - Axelrod, R. (1997). *The Complexity of Cooperation: Agent-Based Models of Competition and Collaboration.* Princeton University Press. — a foundational text on ABMs in the social and behavioral sciences.
    - NetLogo is a widely used, beginner-friendly ABM environment with an extensive model library and its own tutorial: `https://ccl.northwestern.edu/netlogo/`
    - Mesa is the primary Python library for agent-based modeling and a natural choice if you prefer to stay in Python. Documentation and tutorials are at: `https://mesa.readthedocs.io/` and the repository is at: `https://github.com/projectmesa/mesa`
    - The Santa Fe Institute's Complexity Explorer offers free online courses and resources that provide good conceptual grounding: `https://www.complexityexplorer.org/`

### 9.3.6 Linear Ballistic Accumulators

How do we make decisions? We accumulate evidence. That is the idea behind drift diffusion models. And they have been very big over the years, however the math is quite complex and they can be hard to fit. Linear ballistic accumulators are a simplified version. And just as we saw with integrate and fire models simple can sometimes be better, depending on your question.

1. Starting References and Resources

    - Brown, S. D., & Heathcote, A. (2008). The simplest complete model of choice response time: Linear ballistic accumulation. *Cognitive Psychology*, 57(3), 153–178. `https://doi.org/10.1016/j.cogpsych.2007.12.002` — the paper that introduced the LBA model.
    - Donkin, C., Brown, S. D., & Heathcote, A. (2011). Drawing conclusions from choice response time models: A tutorial using the linear ballistic accumulator. *Journal of Mathematical Psychology*, 55(2), 140–151. `https://doi.org/10.1016/j.jmp.2010.10.001` — a tutorial-style companion paper aimed at new users.
    - The `rtdists` R package implements the LBA and related evidence-accumulation models and is a practical starting point for coding: `https://cran.r-project.org/package=rtdists`
    - PyDDM is a Python-based simulator for drift-diffusion and accumulator models that can serve as a useful reference implementation: `https://pyddm.readthedocs.io/`

## 9.4 Project Deliverables Summarized

### 9.4.1 Presentation

A half hour presentation during the last class session that will provide us all with an overview of your topic to include its background, importance, and application. Then you will need to walk us through how to implement and use your computational offering. Lastly, you should address how it is or could be used for better understanding mental or neural questions about the mechanisms of cognition.

### 9.4.2 Report

Your report should be a paper replete with the usual abstract, intro, methods, code, results of simulations, and references. Figures and analyses should be integrated into the manuscript in order to have a reproducible research project. To the extent that this is not felt to be possible you will have to justify it heavily. Tools that you might find useful include Overleaf and Quarto, though you can probably get close with Rmd/Rstudio since that now supports python. If you elect to use another language you may find they have a suitable language for this (e.g. with Racket it is Scribble).

## 9.5 General Suggestions

I will make the groups, and I encourage each of you to find the right role for each member to make the project work efficiently. Some may be stronger writers while others are better coders. All research is collaborative with a mix of talents. Finding out how to discover that, and apportion work will be important in your future endeavors.

We will decide together the deadline for the delivery of the final report.

## 9.6 Some Examples from Last Year

# Backpropagation

PSYCH 420: Group 1 Final Report

Sarah An,
Amanda Clarkson,
Jai Vardhan Gulati,
Nilaethiga Muralitharan





https://github.com/sarahgahyun/backpropagation/tree/main

**Morris-Lecar Networks**

Aurora Hannay, Noah Kim, Leena Abo Al Soud, Avni Gupta, Lily Corkill

PSYCH 420: An Introduction to Computational Neuroscience Methods

Dr. Britt Anderson

April 17th, 2025

`https://colab.research.google.com/drive/1ZbZ2MCkY9Wal8BprVKcyaNC1Cl34Wbqu?usp=sharing`

# 10 Some references

Borsboom, Denny, Han L. J. van der Maas, Jonas Dalege, Rogier A. Kievit, and Brian D. Haig. "Theory Construction Methodology: A Practical

Framework for Building Theories in Psychology." *Perspectives on Psychological Science* 16, no. 4 (February 2021): 756–66. https://doi.org/10.1177/1745691620969647.

Davey, B. A., and H. A. Priestley. *Introduction to Lattices and Order*. Cambridge University Press, 2002. https://doi.org/10.1017/cbo9780511809088.

Eronen, Markus I., and Jan-Willem Romeijn. "Philosophy of Science and the Formalization of Psychological Theory." *Theory &Amp; Psychology* 30, no. 6 (December 2020): 786–99. https://doi.org/10.1177/0959354320969876.

Guest, Olivia, and Andrea E. Martin. "How Computational Modeling Can Force Theory Building in Psychological Science." *Perspectives on Psychological Science*, 2021, 174569162097058. https://doi.org/10.1177/1745691620970585.

Hafez, Omar A., and Allan Gottschalk. "Altered Neuronal Excitability in a Hodgkin-Huxley Model Incorporating Channelopathies of the Delayed Rectifier Potassium Channel." *Journal of Computational Neuroscience* 48, no. 4 (October 2020): 377–86. https://doi.org/10.1007/s10827-020-00766-1.

Hopfield, J J. "Neural Networks and Physical Systems with Emergent Collective Computational Abilities." *Proceedings of the National Academy of Sciences* 79, no. 8 (April 1982): 2554–58. https://doi.org/10.1073/pnas.79.8.2554.

Koch, Tobias, and Alexander Wendt. "Theory Crises in Psychology – What Can We Learn from Structuralism?," April 2025. https://doi.org/10.31219/osf.io/y7xbz_v1.

Lopez, Adele. "Chu Are You?" LessWrong, 2021. https://www.lesswrong.com/posts/89EvBkc4nkbEctzR3/chu-are-you.

Naur, Peter. "Programming as Theory Building." *Microprocessing and Microprogramming* 15, no. 5 (1985): 253–61.

Oberauer, Klaus, and Stephan Lewandowsky. "Addressing the Theory Crisis in Psychology." *Psychonomic Bulletin & Review* 26, no. 5 (2019): 1596–1618.

Oude Maatman, Freek. "Psychology's Theory Crisis, and Why Formal Modelling Cannot Solve It," July 2021. https://doi.org/10.31234/osf.io/puqvs.

Rooij, Iris van, and Giosuè Baggio. "Theory before the Test: How to Build High-Verisimilitude Explanatory Theories in Psychological Science." *Perspectives on Psychological Science*, 2021, 174569162097060. https://doi.org/10.1177/1745691620970604.

Sanbonmatsu, David M., Becky Neufeld, and Steven S. Posavac. "There Is

No Theory Crisis in Psychological Science." *Journal of Theoretical and Philosophical Psychology*, January 2025. https://doi.org/10.1037/teo0000301.

Thagard, Paul. "Explanatory Coherence." *Behavioral and Brain Sciences* 12, no. 3 (September 1989): 435–67. https://doi.org/10.1017/s0140525x00057046.

Tull, Sean. "Monoidal Categories for Formal Concept Analysis," 2020. https://arxiv.org/abs/2012.08268.

Turing, Alan. "On Computable Numbers, with an Application to the Entscheidungsproblem (1936)." In *The Essential Turing*, 58–90. Oxford University PressOxford, 2004. https://doi.org/10.1093/oso/9780198250791.003.0005.

Wille, Rudolf. "Formal Concept Analysis as Mathematical Theory of Concepts and Concept Hierarchies." In *Formal Concept Analysis*, 1–33. Springer Berlin Heidelberg, 2005. https://doi.org/10.1007/11528784_1.